Желілік бағдарламалау

15 апталық дәрістер жинағы

Мазмұны және тақырыптық жоспар

Әр аптаға 1 академиялық сағат бөлінген. Жинақтағы тақырыптар бірізді меңгеруге арналған. Тақырып атауын басып, қажетті дәріске өтуге болады.

Апта	Дәріс тақырыбы	Бет
1	Желілік бағдарламалауға кіріспе	4
2	IP мекенжайлау DNS және порттар	7
3	TCP сокеттері және алғашқы клиент пен сервер	10
4	UDP датаграммалары және қолданба деңгейіндегі сенімділік	13
5	Қолданбалы хаттама және хабарлама шекарасы	16
6	Бірнеше клиент және ағындар арқылы қатар орындау	19
7	Асинхронды желілік бағдарламалау	22
8	HTTP хаттамасы және веб клиент	25
9	REST қағидалары және JSON веб қызметі	28
10	Нақты уақыттағы байланыс WebSocket және SSE	31
11	TLS және желілік қолданбаның қауіпсіздігі	34
12	Үзілістер уақыт шектеуі және қайталау саясаты	37
13	Файлдарды тасымалдау және деректердің тұтастығы	40
14	Желілік бағдарламаларды тестілеу және диагностика	44
15	Желілік қызметті біріктіру және қорытынды жоба	48

Пән туралы мәлімет

Бұл оқу материалы желі арқылы дерек алмасатын бағдарламаларды жобалауды, жазуды және тексеруді үйретеді. Дәрістер ақпараттық технологиялар бағытындағы студенттерге арналған. Негізгі мақсат — клиент пен сервердің жұмысын түсініп, дерек шекарасын дұрыс анықтайтын, қателерді өңдейтін және ресурстарды шектейтін шағын желілік қызмет құрастыру.

Курс 15 аптаға жоспарланған. Әр аптаға 1 академиялық сағат, барлығы 15 академиялық сағат дәріс қарастырылған. Бір академиялық сағат 50 минут деп алынған. Мәтіндер оқытушы түсіндірмесіне, кодты талдауға және аудиториядағы есептерге негіз береді. Өздік тапсырмаларды дәрістен кейін орындау ұсынылады.

Алғышарттар: айнымалылар, шарттар, циклдер, функциялар, файлдар, ерекшеліктер және Python тіліндегі негізгі деректер құрылымдары. Желілік тәжірибе талап етілмейді. Бағдарламалар Python 3.12 және одан кейінгі үйлесімді нұсқаларға арналған. Негізгі мысалдар стандартты кітапханамен жұмыс істейді. WebSocket пен TLS жаттығуларында қосымша орта талаптары жеке көрсетілген.

Курсты аяқтаған студент желі деңгейлерінің қызметін түсіндіреді; IP мекенжайы мен портты ажыратады; TCP және UDP клиенттері мен серверлерін жазады; хабарлама пішімін жобалайды; ағындар мен asyncio қолданады; HTTP сұрауларын өңдейді; TLS тексеруін дұрыс баптайды; уақыт шектеуі, журналдау және сынақтар арқылы желілік қызметтің жұмысын бағалайды.

Мысалдарды орындау үшін әр атауы көрсетілген кодты сол атаумен жеке файлға сақтау қажет. Бір дәрістегі тәуелді файлдар бір каталогта орналасады. Сервер мен клиент екі терминалда іске қосылады. Алдымен сервер іске қосылады. Windows жүйесінде python орнына py командасын қолдануға болады. Серверді тоқтату үшін Ctrl+C басылады. Оқу порттары басқа бағдарламада қолданылса, екі жақтағы порт бірдей бос мәнге ауыстырылады.

Жергілікті мысалдар 127.0.0.1 мекенжайына байланады. Бұл бір компьютерде қауіпсіз тәжірибе жасауға жеткілікті. Бірнеше компьютерді қосқанда тыңдау мекенжайы, желіаралық экран ережесі, қатынау құқығы және TLS жеке бапталады. Кітаптағы шағын серверлер оқу логикасын көрсетеді; қызметті пайдаланушыларға ұсыну талаптары 15 дәрісте талқыланады.

1 Дәріс. Желілік бағдарламалауға кіріспе

Дәріс мақсаты: желілік бағдарламаның құрылымын, дерек алмасу жолын және өнімділіктің негізгі өлшемдерін түсіндіру.

Оқу нәтижелері: студент клиент пен сервер рөлдерін ажыратады, желі деңгейлерін байланыстырады, сұраудың өту жолын сипаттайды және кідіріс пен өткізу қабілетін есептейді.

Бағдарламалар арасындағы байланыс

Желілік бағдарламалау — әртүрлі процестердің желі арқылы ақпарат алмасуын ұйымдастыру. Бір компьютердегі екі процесс те IP желісін пайдаланып сөйлесе алады. Қашықтағы процеске функция шақырғандай қатынау ыңғайлы көрінгенімен, желі қосымша мәселелер енгізеді: жауап кешігеді, байланыс үзіледі, хабар қайта жіберіледі, ал екінші тараптың ішкі күйі тікелей көрінбейді. Сондықтан желілік алгоритм тек сәтті нәтижені емес, күту, қате және қайта қосылу жағдайларын қамтиды.

Клиент әрекетті бастайды, сервер белгілі мекенжайдағы сұрауларды күтеді. Бұл атаулар компьютердің қуатын емес, өзара әрекеттесу рөлін сипаттайды. Мобильді қосымша каталог серверіне клиент ретінде қосылады; каталог сервері дерекқорға сұрау жібергенде өзі клиент рөлін атқарады. Бір жүйеде екі рөл қатар кездеседі. Тең дәрежелі архитектурада әр қатысушы ресурсты ұсынуға да, сұрауға да қатысады.

Қызмет келісімі үш сұраққа жауап береді: кімге қосыламыз, қандай хабар жібереміз және нәтижені қалай танимыз? Бірінші сұраққа мекенжай мен порт жауап береді. Екіншісі хабарлама пішімі мен хаттаманы талап етеді. Үшіншісі мәртебе, жауап идентификаторы және қате кодтары арқылы шешіледі. Осы келісім екі жақта бірдей сақталмаса, желі жұмыс істеп тұрса да бағдарлама дұрыс нәтиже бермейді.

Деңгейлер және инкапсуляция

Желі деңгейлері жауапкершілікті бөледі. Қолданбалы деңгей сұраудың мағынасын анықтайды. Тасымалдау деңгейі процестер арасындағы жеткізуді ұйымдастырады. Интернет деңгейі пакетті мекенжай бойынша бағыттайды. Арналық деңгей көрші құрылғылар арасындағы кадр алмасуды орындайды. Әр деңгей төменгі деңгейдің қызметін пайдаланып, өзіне тиесілі тақырып қосады. Бұл әрекет инкапсуляция деп аталады.

TCP IP деңгейі	Негізгі міндеті	Мысал	Бағдарламашы шешетін мәселе
Қолданбалы	Хабардың мағынасы	HTTP, DNS, жеке хаттама	Сұрау мен жауап келісімі
Тасымалдау	Процестер арасындағы жеткізу	TCP, UDP	Байланыс тәсілін таңдау
Интернет	Мекенжайлау және бағыттау	IPv4, IPv6	Қолжетімді мекенжайды таңдау
Арналық	Бір желі бөлігіндегі жеткізу	Ethernet, Wi Fi	Орта мен интерфейсті тексеру

OSI үлгісі жеті деңгейді бөледі. Оның сеанстық және ұсыну міндеттері TCP/IP негізіндегі бағдарламаларда қолданбалы кітапханалармен жиі орындалады. Үлгілерді нақты бір кітапханаға күшпен сәйкестендіру қажет емес: олар қатенің қай жерде пайда болғанын талдауға көмектеседі. Мысалы, атау шешілмесе DNS тексеріледі, TCP қосылымы құрылмаса мекенжай мен порт тексеріледі, HTTP 403 келсе қолданбаның рұқсат ережесі қаралады.

Бір веб сұраудың жолы

Пайдаланушы https://example.org/catalog мекенжайын ашады. Клиент URL құрамынан схема, хост және жолды алады. Атау шешуші хостқа сәйкес IP мекенжайларын іздейді. HTTP/1.1 немесе HTTP/2 қолданылғанда клиент TCP байланысын құрып, HTTPS үшін TLS келісуін орындайды. Одан кейін HTTP сұрауы жіберіледі. Сервер

жауапты қалыптастырады, клиент жауап мәртебесі мен денесін өңдейді. Қосылым мен DNS нәтижесі қайта қолданылса, келесі сұрауда кейбір қадамдар қысқарады.

HTTP нұсқалары бірдей мағынадағы сұрауларды әртүрлі тәсілмен тасымалдайды. HTTP/3 QUIC үстінде жұмыс істейді; QUIC UDP арқылы сенімді ағындар мен қорғалған байланысты ұйымдастырады. UDP қолданылуы қолданбаның барлық дерегі сенімсіз жеткізіледі деген қорытындыға әкелмейді: кепілдіктерді жоғарғы хаттама ұсына алады. [16]

Кідіріс пен өткізу қабілеті

Кідіріс — әрекеттің басталуынан нәтижесіне дейінгі уақыт. RTT — сұрау барып, жауап қайтқанға дейінгі екіжақты уақыт. Өткізу қабілеті уақыт бірлігінде жеткізілген дерек көлемімен өлшенеді. Арнаның атаулы жылдамдығы пайдалы дерек жылдамдығына тең емес: тақырыптар, кезектер, қайта жеткізу және қолданба жұмысы уақыт алады.

Есеп: 10 000 000 байт файл 20 000 000 бит/с арнамен жіберілсін. Дерек көлемін битке ауыстырғанда 80 000 000 бит алынады. Тек жіберуге кететін төменгі уақыт шегі 4 секунд. Оған атау шешу, байланыс орнату және өңдеу уақыты қосылады. Бұл есеп идеал жағдайға арналған; нақты уақытты өлшеу қажет. Көп шағын сұрауда RTT басым әсер етеді, ал үлкен файлда өткізу қабілеті басым рөл атқарады.

URL құрамын талдау мысалы

Төмендегі бағдарлама желіге қосылмайды. Ол пайдаланушы енгізген URL мен қосылым параметрлерінің арасындағы байланысты көрсетеді. Жол сұралатын ресурсты белгілейді; query бөлігі сүзгі мен іздеу параметрлерін тасиды.

week01_url.py

```
from urllib.parse import urlsplit, parse_qs

url = "https://example.org:8443/catalog?page=2&lang=kk"
parts = urlsplit(url)
port = parts.port or (443 if parts.scheme == "https" else 80)

print("scheme:", parts.scheme)
print("host:", parts.hostname)
print("port:", port)
print("path:", parts.path)
print("query:", parse_qs(parts.query))
```

Іске қосу: python week01_url.py. Күтілетін мәндер: https, example.org, 8443, /catalog және page пен lang кілттері бар сөздік. parse_qs әр кілтке мәндер тізімін береді, себебі URL ішінде бір параметр бірнеше рет кездесе алады. Қате URL тексеруі кейінгі клиентте қосылуға дейін орындалуы тиіс.

Қателерді талдау және бекіту

Жиі қате — сервер жауап бермесе бірден «интернет істемейді» деу. Мәселені қабаттарға бөліп тексеріңіз: жергілікті интерфейс, бағыт, атау шешу, порт, TLS, қолданба мәртебесі. Тағы бір қате — HTTP қатесін көлік қатесімен теңестіру. 404 жауабы сервермен байланыс орнағанын және сұраудың қолданбаға жеткенін көрсетеді.

Бақылау сұрақтары

1. Бір процесс қашан клиент, қашан сервер рөлін атқарады?
2. Хаттама келісімі қандай үш мәселені шешеді?

3. RTT мен өткізу қабілетінің айырмасы қандай?

4. HTTP 404 пен TCP қосылу қатесін қалай ажыратасыз?

Өздік жұмыс: білім беру порталындағы «тапсырманы жүктеу» әрекетінің өту жолын мәтінмен сипаттаңыз. Әр қадамға бір ықтимал ақау мен оны тексеру әрекетін жазыңыз. Қосымша есеп: 25 000 000 байт файлды 50 000 000 бит/с арнамен жіберудің идеал уақытын есептеңіз. Нәтиже: 4 секунд; қосымша кідірістерді бөлек атаңыз.

Дәрісті өткізу реті: рөлдер мен мәселе қоюға 15 минут, деңгейлерге 25 минут, сұрау жолы мен есепке 25 минут, кодқа 15 минут, бекітуге 10 минут.

2 Дәріс. IP мекенжайлау DNS және порттар

Дәріс мақсаты: желілік қызметке жету үшін қолданылатын мекенжайлау мен атау шешу тетіктерін меңгеру.

Оқу нәтижелері: студент IPv4 пен IPv6 пішімдерін оқиды, желі префиксін есептейді, DNS жазбаларын ажыратады және Python арқылы қолжетімді мекенжайларды алады.

IPv4 және желі префиксі

IPv4 мекенжайы 32 биттен тұрады және әдетте төрт ондық санмен жазылады. Әр бөлік 0–255 аралығында орналасады. CIDR жазылуындағы /24 алғашқы 24 биттің желі префиксіне тиесілі екенін көрсетеді. Қалған биттер ішкі желідегі мекенжайларды айқындайды. Бағыттаушы пакет баратын мекенжайды өз бағыттау кестесіндегі префикстермен салыстырып, ең ұзын сәйкес префиксті таңдайды.

192.168.10.34/27 мысалында маска 255.255.255.224, блок көлемі 32 мекенжай. 34 саны 32–63 блогына кіреді. Желі мекенжайы 192.168.10.32, таратылым мекенжайы 192.168.10.63, кәдімгі ішкі желідегі хосттар аралығы 192.168.10.33–192.168.10.62. Бұл ережені /31 нүктеден нүктеге қосылымына сол күйінде қолдануға болмайды; оның мекенжай пайдалану тәртібі бөлек.

Жеке мекенжайлар ұйымның ішкі желісінде қолданылады. Олардың интернетте тікелей бағытталуын күтуге болмайды. NAT ішкі және сыртқы мекенжайлар мен порттардың сәйкестігін жүргізе алады. NAT шифрлау немесе қолданбаға рұқсат беру тетігін алмастырмайды. Бір компьютердегі localhost тәжірибесінде пакет сыртқы желіге шығуды талап етпейді, сондықтан маршрутизатор ақауы бұл тәжірибеге әсер етпеуі мүмкін.

IPv6 және жергілікті байланыс

IPv6 мекенжайы 128 биттен тұрады. Он алтылық топтар қос нүктемен бөлінеді; қатар тұрған нөл топтарын бір рет :: арқылы қысқартуға рұқсат етіледі. Мысалы, 2001:db8:0:0:0:0:0:10 мекенжайы 2001:db8::10 түрінде жазылады. 2001:db8::/32 оқу құжаттарындағы мысалдарға арналған. IPv6 ішінде IPv4 үлгісіндегі таратылым жоқ; оның орнына тиісті көпадрестік тетіктер қолданылады. [7]

127.0.0.1 — IPv4 loopback мысалы, ::1 — IPv6 loopback мекенжайы. Сервердегі 0.0.0.0 байланыстырылатын IPv4 интерфейстерінің барлығын білдіреді; клиент қосылатын нақты қашықтағы хост мекенжайы ретінде қолданылмайды. IPv6 серверінде :: мәнінің IPv4 қосылымдарын да қабылдауы операциялық жүйе мен сокет параметрлеріне тәуелді. Екі стекке қолдау талап етілсе, оны бөлек тексеру керек.

Порт және қосылымды сәйкестендіру

Порт тасымалдау деңгейіндегі қызмет нүктесін белгілейді. TCP мен UDP порт кеңістіктері бөлек: бір санды екі хаттамада әртүрлі қызмет пайдалана алады. Белсенді TCP қосылымы жергілікті IP, жергілікті порт, қашықтағы IP және қашықтағы порт арқылы ажыратылады. Сондықтан бір сервердің 443 порты мыңдаған клиентпен қатар жұмыс істей алады; клиент жақтағы параметрлер әртүрлі.

Мән	Мақсаты	Бағдарламадағы қолданылуы
127.0.0.1	Осы компьютерге IPv4 арқылы қатынау	Жергілікті зертхана
::1	Осы компьютерге IPv6 арқылы қатынау	IPv6 сынағы
0.0.0.0	Барлық жергілікті IPv4 интерфейстері	Сервердің bind параметрі
8000 немесе 9000	Курста таңдалған оқу порты	Клиент пен серверде бірдей беріледі
0	Жүйеге бос порт таңдауды тапсыру	Автоматты сынақтағы bind

Клиент bind шақырмай қосылса, операциялық жүйе жергілікті портты өзі таңдайды. Қолданбаның портын IP мекенжаймен шатастыру портқа емес, қате қызметке қатынауға әкеледі. «Address already in use» қатесі

қызметтің осы мекенжай мен портты иеленіп тұрғанын немесе қайта байланысуға байланысты шектеуді көрсетуі мүмкін; нақты себеп жүйе хабарламасынан анықталады.

DNS жұмысы және кэш

DNS домендік атауды ресурс жазбаларымен байланыстырады. A жазбасы IPv4 мекенжайын, AAAA IPv6 мекенжайын, CNAME басқа атауға сілтемені, MX пошта қабылдау бағытын сипаттайды. Клиент әдетте рекурсивті резолверге жүгінеді. Резолвер жауапты кэштен табады немесе домен иерархиясы бойынша іздейді. TTL жазбаны кэште сақтау мерзімін шектейді; өзгеріс барлық клиентке бір сәтте жетпейді. [6]

Бағдарлама бір атауға тек бір IPv4 мекенжайы бекітілген деп қабылдамауы тиіс. DNS бірнеше адрес қайтара алады, ал операциялық жүйе атауды жергілікті hosts файлы сияқты басқа тетіктермен де шеше алады. getaddrinfo жүйенің атау шешу интерфейсін пайдаланады. Нәтижелерді ретімен байқап қосылатын create_connection қарапайым TCP клиентіне ыңғайлы. DNS нәтижесі сайттың түпнұсқалығын жеке өзі дәлелдемейді; HTTPS кезінде сертификат тексеруі қосымша қажет.

Мекенжайларды бағдарламада зерттеу

week02_addresses.py

```
import ipaddress
import socket

net = ipaddress.ip_network("192.168.10.34/27", strict=False)
hosts = list(net.hosts())
print("network:", net.network_address)
print("broadcast:", net.broadcast_address)
print("hosts:", hosts[0], hosts[-1], len(hosts))

seen = set()
for family, kind, proto, name, address in socket.getaddrinfo(
    "localhost", 9000, type=socket.SOCK_STREAM
):
    key = (family, address)
    if key not in seen:
        seen.add(key)
        print(family.name, address)
```

Іске қосу: python week02_addresses.py. Бірінші бөлікте 192.168.10.32, 192.168.10.63 және 30 хост шығады. Екінші бөліктің жол саны мен реті жүйенің IPv4 және IPv6 баптауына тәуелді. strict=False хост биттері бар жазбадан желіні алуға рұқсат береді. Үлкен IPv6 желісінің барлық хостын list түріне айналдыруға болмайды: мекенжайлар саны жад көлемінен асып түседі. Бұл мысалда шағын /27 ғана қолданылады.

Бекіту

Атау шешілмеуі, маршруттың жоқтығы және порттың жабықтығы — әртүрлі жағдайлар. Алдымен атаудың қандай адрестерге шешілгенін қараңыз. Содан кейін нақты адрес пен портқа қосылуды тексеріңіз. Бір адрес істеп, екіншісі істемесе, IPv4 пен IPv6 баптауларын салыстырыңыз. Осындай реттілік мәселені кездейсоқ параметр ауыстырудан тезірек оқшаулайды.

Бақылау сұрақтары

1. /27 префиксі нені көрсетеді?
2. TCP 9000 мен UDP 9000 бір ресурс па?
3. 0.0.0.0 мен 127.0.0.1 қолданылуы қалай ажыратылады?

4. DNS TTL өзгерістің таралуына қалай әсер етеді?

Өздік жұмыс: 192.168.5.77/26 желісінің желі мекенжайын, таратылымын және хост аралығын есептеңіз. Тексеру мәндері: .64, .127 және .65–.126. localhost үшін алынған мекенжайларды кестеге түсіріп, жүйеңізде IPv6 бар-жоғын негіздеңіз.

Дәрісті өткізу реті: IPv4 есебіне 25 минут, IPv6 мен порттарға 20 минут, DNS түсіндірмесіне 20 минут, код талдауға 15 минут, бекітуге 10 минут.

3 Дәріс. TCP сокеттері және алғашқы клиент пен сервер

Дәріс мақсаты: TCP сокетінің өмірлік циклін меңгеріп, екі бағдарлама арасында байт алмасуды іске асыру.

Оқу нәтижелері: студент bind, listen, accept, connect, sendall және recv міндеттерін түсіндіреді; TCP арқылы хабар жібереді; ресурстарды жабады және уақыт шектеуін орнатады.

TCP ұсынатын қызмет

TCP қосылған екі нүктенің арасында реттелген байт ағынын ұсынады. Ол жоғалған бөліктерді қайта жеткізу, қайталанған бөліктерді ажырату және қабылдау ретін қалпына келтіру тетіктерін қолданады. Алайда TCP қолданба хабарламаларының шекарасын сақтамайды. Бір sendall шақыруындағы байттар бірнеше recv шақыруына бөлінуі немесе бірнеше жіберілім бір recv нәтижесіне бірігуі мүмкін. [3]

TCP жеткізуінің сәтті жүруі қашықтағы қолданбаның бизнес әрекетін аяқтағанын дәлелдемейді. sendall қатесіз қайтса, байттар жергілікті желілік стекке берілгенін білеміз; төлемнің тіркелгенін не файлдың тұрақты сақталғанын білмейміз. Мұндай растауды қолданбалы жауаппен анықтаймыз. Желі үзілген сәтте сұраудың орындалған-орындалмағаны белгісіз қалуы ықтимал, сондықтан кейінгі дәрістерде операция идентификаторлары қарастырылады.

Сервер сокеті мен қабылданған сокет

socket(AF_INET, SOCK_STREAM) IPv4 TCP сокетін жасайды. bind оны жергілікті мекенжай мен портқа бекітеді. listen кіріс қосылымдарын күту режимін іске қосады. accept жаңа клиентпен сөйлесуге арналған бөлек сокет пен клиент мекенжайын қайтарады. Тыңдаушы сокет басқа клиенттерді қабылдау үшін сақталады; дерек алмасу accept қайтарған сокет арқылы өтеді. [1]

listen ішіндегі кезек мөлшерін белсенді клиенттердің нақты саны деп түсіндіруге болмайды. Ол операциялық жүйе басқаратын қабылдануды күткен қосылымдар кезегіне қатысты, ал жүйе мәнді өзгертуі немесе шектеуі мүмкін. Бір клиентті өңдейтін циклдегі ұзақ күту келесі клиенттің қабылдануын тежейді. Бұл архитектураның шектеуі, TCP қатесі емес.

Клиентке көбіне bind қажет емес. create_connection хост пен портқа қосылып, сәтті сокет қайтарады. timeout күту шегін береді. Атау шешуге кететін уақыттың барлық платформада дәл осы шекпен қамтылатынына сенуге болмайды; қатаң жалпы мерзім қажет жүйеде барлық кезең жеке бақыланады. with блогы қалыпты аяқталғанда да, ерекшелік шыққанда да сокетті жабады.

Жіберу және қабылдау

send тек байттардың бір бөлігін жіберуі мүмкін және жіберілген санды қайтарады. sendall барлық берілген байтты жіберуге тырысады немесе ерекшелік көтереді. recv(4096) ең көбі 4096 байт оқиды; ол осы көлемнің толуын міндетті түрде күтпейді. TCP сокетіндегі recv нәтижесі b"" болса, қарсы жақ жіберу бағытын жабып, оқылатын дерек аяқталған. [2]

Келесі echo сервер әр алынған байт бөлігін өзгеріссіз қайтарады. Клиент қысқа мәтін жібереді және дәл сол хабар ұзындығына жеткенше жауап жинайды. Мұнда ұзындық бастапқы жіберілген деректен белгілі. Белгісіз көлемдегі хабарларда арнайы шекара ережесі қажет; ол 5 дәрісте қарастырылады.

week03_server.py

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.bind(("127.0.0.1", 9000))
```

```
    server.listen(5)
    print("Listening on 127.0.0.1:9000")
    while True:
        conn, address = server.accept()
        with conn:
            conn.settimeout(10)
            try:
                while True:
                    data = conn.recv(4096)
                    if not data:
                        break
                    conn.sendall(data)
            except (TimeoutError, ConnectionError) as error:
                print(address, type(error).__name__)
```

week03_client.py

```
import socket

message = "Сәлем, желі!".encode("utf-8")
try:
    with socket.create_connection(("127.0.0.1", 9000), timeout=5) as sock:
        sock.sendall(message)
        result = bytearray()
        while len(result) < len(message):
            part = sock.recv(len(message) - len(result))
            if not part:
                raise EOFError("Incomplete echo")
            result.extend(part)
        print(result.decode("utf-8"))
except (OSError, EOFError) as error:
    print(type(error).__name__, str(error))
```

Бірінші терминал: python week03_server.py. Екінші терминал: python week03_client.py. Клиент «Сәлем, желі!» мәтінін шығарады. Сервер ашық қалып, жаңа клиентті күтеді. Клиенттегі len(message) таңба санын емес, UTF-8 байт санын береді. Қазақ әріптерінің байт саны мен таңба санын тең деп алу қабылдауды ерте тоқтатуы мүмкін.

Байланысты аяқтау және қателер

close сокет ресурсын босатады. shutdown(SHUT_WR) жіберу бағытының аяқталғанын білдіруге қолданылады, бірақ қабылдау бағытын ашық қалдыра алады. Егер хаттама жауаптың соңын қосылымның жабылуымен белгілесе, клиент осы ережені алдын ала білуі тиіс. Әр сұраудан кейін жабу қарапайым, бірақ жиі байланыс құру қосымша уақыт алады. Бір қосылыммен бірнеше хабар алмасу үшін хабар шекарасы қажет.

ConnectionRefusedError көбіне көрсетілген портта тыңдаушы жоқ екенін көрсетеді. TimeoutError уақыт шегінде әрекет аяқталмағанын көрсетеді; ол жеке өзі сервердің істен шыққанын дәлелдемейді. ConnectionResetError қосылымның күрт үзілгенін көрсетеді. Әр жағдайды журналда бөлек атау кейін диагностикаға көмектеседі. Барлық ерекшелікті үндемей басу жұмыстың тоқтау себебін жасырады.

Бақылау сұрақтары

1. accept қайтарған сокет тыңдаушыдан несімен ерекшеленеді?
2. Бір recv неге барлық хабарды бермейді?
3. sendall нені растайды?
4. TCP ішінде b"" нені білдіреді?

Өздік жұмыс: клиент хабарын 100 000 байтқа үлкейтіп, қабылданған байт санын тексеріңіз. Серверді қоспай клиентті орындап, қате атауын жазыңыз. Кейін сервердің қабылдау буферін 17 байтқа ауыстырып, хабар мазмұны өзгермейтінін тексеріңіз. Қабылдау шақыруларының саны желі мен жүйеге тәуелді екенін түсіндіріңіз.

Дәрісті өткізу реті: TCP қасиеттеріне 20 минут, сокет цикліне 20 минут, бағдарламаларды талдауға 30 минут, қателерге 10 минут, бекітуге 10 минут.

4 Дәріс. UDP датаграммалары және қолданба деңгейіндегі сенімділік

Дәріс мақсаты: UDP арқылы хабар алмасуды құрып, жоғалу, қайталану және уақыт шектеуі мәселелерін қолданба деңгейінде шешу.

Оқу нәтижелері: студент TCP мен UDP айырмасын негіздейді, sendto және recvfrom қолданады, сұрау идентификаторын тексереді және шектелген қайталау алгоритмін түсіндіреді.

UDP қызметінің ерекшелігі

UDP жеке датаграммаларды тасымалдайды. Бір жіберілген датаграмма жеткізілсе, қабылдаушы оны хабар ретінде оқиды. UDP жеткізуді, реттілікті және қайталанудан қорғауды өзі қамтамасыз етпейді. Қолданба бұл талаптар керек болса, қосымша тетіктер енгізеді. UDP тақырыбы 8 байттан тұрады: бастапқы порт, мақсат порт, ұзындық және бақылау қосындысы өрістері бар. [4]

UDP таңдау тек «жылдам» деген бір сипатқа сүйенбейді. Датчиктің әр секундтағы күйі үшін ескі өлшемді қайта жіберуден жаңа өлшемді жеткізу пайдалырақ. Ал құжаттың бір бөлігі жоғалса, файл жарамсыз қалады. Бірінші жағдайда қолданба жоғалтуды қабылдай алады; екінші жағдайда толықтық пен рет талап етіледі. TCP, UDP немесе UDP үстіндегі дайын сенімді хаттама осы талаптарға сәйкес таңдалады.

UDP ішінде connect шақыруға болады, бірақ ол TCP тәрізді байланыс орнату келісуін орындамайды. Ол әдетте әдепкі қашықтағы мекенжайды бекітіп, send және recv қолдануға мүмкіндік береді. Мұндай сокетке келетін дерек таңдалған тараппен шектеледі. Дегенмен мекенжайды тексеру криптографиялық аутентификацияны алмастырмайды.

Датаграмма өлшемі және буфер

UDP хабар шекарасын сақтағанымен, қабылдау буфері қысқа болса хабардың қалған бөлігі жоғалуы немесе платформаға тән қате туындауы мүмкін. Сондықтан хаттама рұқсат етілетін ең үлкен хабар көлемін бекітіп, қабылдаушы жеткілікті буфер бөледі. TCP ішінде шағын буфер қалған байттарды кейін оқуға мүмкіндік береді; UDP ішінде осы ережеге сүйенуге болмайды.

Үлкен IP пакеті жолдағы MTU шегіне сыймаса, бөлшектеу немесе жеткізбеу мәселесі туындайды. Бір фрагменттің жоғалуы бүкіл датаграмманың жарамсыз қалуына әкеледі. Қолданба шағын хабарларды ұстанып, жол MTU ерекшеліктерін ескергені дұрыс. Курс мысалындағы 1000 байт шегі оқу келісімі ретінде таңдалған; оны кез келген желі үшін әмбебап кепіл деп қабылдауға болмайды. UDP қызметі жіберу қарқыны мен желі кептелісін де ескеруі тиіс. [19]

Уақыт шектеуі және сұрау идентификаторы

Клиент жауап келмесе белгілі уақыттан кейін қайталайды. Бірден шексіз қайталау ақаулы қызметке жүктемені арттырады. Шектелген әрекет саны, ұлғаятын күту аралығы және жалпы мерзім керек. Кешіккен ескі жауап жаңа сұрауға қабылданбауы үшін әр сұраудың идентификаторы болады. ЖауДәріс. сол идентификатор қайтарылады.

Қайта жіберу операцияны екі рет орындау қаупін енгізеді. Күйді тек оқу немесе echo қайтару қайталанса, нәтижесі өзгермейді. Ал «есептегішті бірге өсір» әрекетін қайталау есепті бұзады. Осындай операцияға сервер сұрау идентификаторы бойынша бұрынғы нәтижені сақтап, қайталанған сұрауға сол нәтижені қайтарады. Кэштің мерзімі, сыйымдылығы және қайта іске қосылғандағы мінезі хаттамаға кіреді.

UDP echo мысалы

week04_server.py

```
import socket

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
    sock.bind(("127.0.0.1", 9001))
    print("UDP listening on 127.0.0.1:9001")
    while True:
        data, address = sock.recvfrom(65535)
        if 0 < len(data) <= 1000:
            sock.sendto(data, address)
```

week04_client.py

```
import secrets
import socket
import time

token = secrets.token_hex(8).encode("ascii")
request = token + b":hello"
with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as sock:
    sock.connect(("127.0.0.1", 9001))
    for attempt in range(3):
        deadline = time.monotonic() + 0.5 * (2 ** attempt)
        sock.send(request)
        response = None
        while time.monotonic() < deadline:
            sock.settimeout(max(0.001, deadline - time.monotonic()))
            try:
                data = sock.recv(65535)
            except TimeoutError:
                break
            if data == request:
                response = data
                break
        if response is not None:
            print("reply:", response.decode("ascii"))
            break
    else:
        print("No valid reply after 3 attempts")
```

Іске қосу: бірінші терминалда python week04_server.py, екіншісінде python week04_client.py. Қалыпты нәтиже — кездейсоқ идентификатор мен hello жолы. time.monotonic өткен уақытты өлшейді; жүйе сағаты түзетілсе де мерзімді есептеу бұзылмайды. Әр әрекеттегі жауаптың мазмұны тексеріледі. Сервер жоқ кезде кейбір жүйелер TimeoutError орнына ConnectionRefusedError бере алады; оны толық клиентте OSError тармағымен өңдеу қажет.

Қауіпсіз және бақыланатын тәжірибе

Жоғалуды тәжірибеде байқау үшін сервердегі әр екінші датаграммаға жауап бермеуге арналған санауыш енгізіңіз. Бұл тек зертханадағы жасанды жағдай. Клиенттің әрекет саны мен күту уақытын журналға түсіріңіз. Қайталау саны өскенде орташа кідіріс өсетінін түсіндіріңіз. Бір хостта жоғалу байқалмауы UDP кепілдігі бар екенін дәлелдемейді; зертхана желісі жай ғана бұл жағдайды тудырмаған.

UDP сервері шағын сұрауға үлкен жауап берсе, шағылдыру арқылы жүктеме күшейту қаупін туғызады. Бұл курста сервер тек алынған қысқа хабарды қайтарады және жергілікті адреске байланады. Нақты қызметте сұрауды тексеру, жауап көлемін шектеу және жіберушіге байланысты қарқын шегі қарастырылады.

Бақылау сұрақтары

1. UDP connect неге TCP келісуіне тең емес?
2. Қысқа қабылдау буфері неге қауіпті?
3. Сұрау идентификаторы қандай қатені болдырмайды?
4. Қайталанатын күй өзгертуші әрекетті қалай қорғайсыз?

Өздік жұмыс: сервер әр екінші хабарды елемейтін нұсқаны жасап, бес клиент сұрауының нәтижесін тіркеңіз. Кестеге идентификатор, әрекет саны және өткен уақытты жазыңыз. Қосымша тапсырма: күй өзгертпейтін температура сұрауы үшін хабар пішімін құрып, қате жауап пен кеш жауапты ажырату ережесін жазыңыз.

Дәрісті өткізу реті: UDP қасиеттеріне 20 минут, өлшем мен жоғалуға 20 минут, қайталау алгоритміне 20 минут, мысалға 20 минут, бекітуге 10 минут.

5 Дәріс. Қолданбалы хаттама және хабарлама шекарасы

Дәріс мақсаты: TCP ағынынан жеке хабарламаларды бөліп алатын келісім мен қайта қолданылатын код құрастыру.

Оқу нәтижелері: студент шекаралау тәсілдерін салыстырады, UTF-8 пен JSON қолданады, ұзындық префиксін өңдейді және кіріс деректің көлемі мен құрылымын тексереді.

Хабарлама шекарасын таңдау

TCP ағыны байттардан тұрады; recv нәтижесін дайын хабар деп қабылдау қолданбаны желі жағдайына тәуелді етеді. Хабар шекарасын анықтаудың төрт кең тараған тәсілі бар: тұрақты ұзындық, бөлгіш таңба, ұзындық префиксі және қосылымды жабу. Тұрақты ұзындық өлшеу құрылғысының бірдей жазбаларына жарайды. Жаңа жолмен аяқтау мәтіндік командаларға ыңғайлы, бірақ бөлгішті хабар ішіне енгізу тәртібі қажет. Ұзындық префиксі еркін байттық денені тасымалдауға мүмкіндік береді.

Тәсіл	Артықшылығы	Ескерілетін шектеу
Тұрақты ұзындық	Оқу ережесі қарапайым	Ұзындығы өзгеретін хабарға тиімсіз
Жаңа жолмен бөлу	Қолмен оқуға ыңғайлы	Жол ұзындығы мен escaping ережесі керек
Ұзындық префиксі	Мәтін мен бинарлық дерекке жарайды	Тақырыпты да циклмен оқу қажет
Қосылымның аяқталуы	Бір хабарға қарапайым	Сол қосылыммен келесі хабар жіберілмейді

Шекаралау мен сериализация бөлек міндет атқарады. Шекаралау хабардың қай жерден басталып, қай жерде аяқталатынын анықтайды. Сериализация Python сөздігін немесе басқа құрылымды байттарға түрлендіреді. JSON мәтінінің өзі TCP ішінде хабар соңын білдірмейді. Қабылдаушы алдымен бүтін денені жинап, содан кейін JSON талдауын орындайды.

Мәтін кодтауы және дерек моделі

Python str таңбалар тізбегін, bytes байттар тізбегін көрсетеді. Желілік сокет bytes қабылдайды. Мәтін encode("utf-8") арқылы байтқа, толық мәтіндік дене decode("utf-8") арқылы жолға айналады. Бір көпбайтты таңба екі recv нәтижесіне бөлінсе, әр бөлікті жеке decode жасау қате туғызады. Бүтін хабарды жинау немесе ағындық декодер қолдану бұл мәселені шешеді.

JSON нысан, массив, жол, сан, логикалық мән және null деректерін береді. Желі арқылы алмасуда UTF-8 келісімі қолданылады. JSON ішіне NaN және Infinity стандартты сан ретінде кірмейді. Python сериализациясында allow_nan=False параметрі мұндай мәндерді жібермейді. JSON құрылымы дұрыс оқылғанымен, оның өрістері қолданбаның келісіміне сәйкес тексерілуі тиіс. [18]

Оқу хаттамасы әр хабарды 4 байттық таңбасыз ұзындықпен бастайды. !I пішімі 32 биттік бүтін санды желілік байт ретімен кодтайды. Одан кейін көрсетілген ұзындықтағы UTF-8 JSON денесі келеді. Рұқсат етілген дене көлемі 65 536 байт. Нөлдік немесе шектен үлкен ұзындық бірден қабылданбайды. Ұзындыққа сеніп жад бөлу ресурсты тауыса алады.

Қайта қолданылатын хаттама модулі

protocol.py

```
import json
import struct
```

```
MAX_BODY = 65536

def recv_exact(sock, count):
    data = bytearray()
    while len(data) < count:
        part = sock.recv(count - len(data))
        if not part:
            raise EOFError("Connection ended inside a frame")
        data.extend(part)
    return bytes(data)

def reject_constant(value):
    raise ValueError("Non-standard JSON number: " + value)

def encode_message(message):
    if not isinstance(message, dict):
        raise ValueError("JSON object required")
    body = json.dumps(
        message, ensure_ascii=False, allow_nan=False
    ).encode("utf-8")
    if not 0 < len(body) <= MAX_BODY:
        raise ValueError("Invalid frame size")
    return struct.pack("!I", len(body)) + body

def send_message(sock, message):
    sock.sendall(encode_message(message))

def recv_message(sock):
    size = struct.unpack("!I", recv_exact(sock, 4))[0]
    if not 0 < size <= MAX_BODY:
        raise ValueError("Invalid frame size")
    body = recv_exact(sock, size)
    message = json.loads(
        body.decode("utf-8"), parse_constant=reject_constant
    )
    if not isinstance(message, dict):
        raise ValueError("JSON object required")
    return message
```

recv_exact алдымен тақырыпты, кейін денені толық жинауға қолданылады. Ол сокеттен сұралған мөлшерден артық оқымайды, сондықтан келесі хабар байттарын жұтып қоймайды. Кодтағы EOFError бүтін хабар аяқталмай қосылым жабылғанын білдіреді. Хабар шекарасындағы қалыпты жабылуды бөлек ажыратын күрделі серверге тақырыпты оқу алдында «дерек жоқ» күйін қайтару ережесі қосылады.

Келісімнің мағынасы мен нұсқасы

Дене мысалы: {"v": 1, "id": "q17", "op": "echo", "text": "Сәлем"}. v — келісім нұсқасы, id — сұрау мен жауапты байланыстыратын белгі, op — операция, text — параметр. ЖауДәріс. сол id және ok өрісі қайтарылады. Қате болса error тұрақты машиналық кодты береді; пайдаланушыға арналған түсіндірме бөлек өрісте сақталады. Қате мәтінін машиналық шешімнің жалғыз негізі етуге болмайды, себебі мәтін аударылуы немесе өзгеруі мүмкін.

Нұсқа енгізу кейінгі өзгерістерді басқаруға көмектеседі. Сервер қолдамайтын нұсқаға анық жауап береді. Міндетті өрісті алып тастау ескі клиентті бұзады; қосымша міндетті емес өрісті енгізу кей келісімде үйлесімді. Белгісіз өрістерді елемеу немесе қабылдамау тәртібі алдын ала жазылады. Бірдей JSON кілтінің қайталануын қабылдау да жеке шешім: қатаң қызмет мұны арнайы object_pairs_hook арқылы кері қайтара алады.

Шекараларды есептеу және тексеру

Есеп: дене 300 байт болса, толық кадр 304 байт. Тақырыптың он алтылық көрінісі 00 00 01 2c. Бұл сан мәтіндегі "300" үш таңбасына тең емес; ол төрт байттық бинарлық сан. Бір тақырыптың 2 байты, кейін қалған 2 байты келсе де recv_exact дұрыс жинайды. Екі кадр бір қабылдау буферіне келсе де әр recv_message өз кадрын ғана оқиды.

Модульді protocol.py атымен сақтаңыз. Ол 6 және 14 дәрістердегі мысалдарға қажет. Бұл файлды жеке іске қосқанда мәтін шықпайды: ол басқа бағдарламалар шақыратын функцияларды анықтайды. Уақыт шектеуі модуль ішінде орнатылмайды; сокетті жасаған тарап оны орнатады. Бір recv мерзімі мен бүкіл хабардың жалпы мерзімі әртүрлі екенін есте ұстаңыз.

Бақылау сұрақтары

1. JSON неге хабар шекарасын жеке өзі шешпейді?
2. Ұзындықты қай сәтте тексеру керек?
3. Желілік байт реті не үшін қажет?
4. Қате коды мен қате мәтіні қалай ажыратылады?

Өздік жұмыс: файл жіберу метадерегіне арналған JSON келісімін жазыңыз. Өрістерге v, id, filename, size және sha256 енгізіп, әрқайсысының пішімі мен шегін көрсетіңіз. Бос дене, артық көлем, UTF-8 қатесі және белгісіз нұсқа үшін сервер әрекетін анықтаңыз. Күтілетін нәтиже — басқа студент клиент жаза алатын бір беттік хаттама сипаттамасы.

Дәрісті өткізу реті: шекаралауға 20 минут, кодтау мен JSON-ға 20 минут, модуль талдауына 30 минут, келісімге 10 минут, бекітуге 10 минут.

6 Дәріс. Бірнеше клиент және ағындар арқылы қатар орындау

Дәріс мақсаты: бірнеше клиентке қызмет көрсететін архитектураны құрып, ортақ күй мен ресурстарды басқаруды түсіндіру.

Оқу нәтижелері: студент тізбекті сервердің шектеуін сипаттайды, жұмысшы ағындар мен шектелген кезек қолданады, жарысу жағдайын таниды және бұғаттау аймағын дұрыс таңдайды.

Тізбекті өңдеуден қатар өңдеуге өту

Тізбекті сервер бір клиенттің сұрауын аяқтағанша келесісін өңдемейді. Клиент деректі баяу жіберсе, басқа клиенттерге де кідіріс тарайды. Қатар орындау бірнеше әрекеттің уақыт аралығында қабаттасуын қамтамасыз етеді. Параллель орындау бірнеше есептің бір сәтте әртүрлі процессор өзегінде жүруін білдіреді. Желіде күту көп болғандықтан, қатар орындаудың өзі қызмет сапасын жақсартады.

Әр қосылымға бір ағын жасау түсінікті, бірақ шектеусіз ағын саны жад пен жоспарлау шығынын өсіреді. Тұрақты жұмысшы пулы ағын санын шектейді. Кіріс кезек қысқа уақыттағы жүктеме ауытқуын сіңіреді, бірақ шексіз кезек мәселені жадқа ауыстырады. Жұмысшылар да, кезек те шектеліп, кезек толғанда қабылдау саясаты анықталады.

Ағын процесс және ортақ күй

Ағындар бір процестің жадын бөліседі. Сондықтан ортақ сөздікке немесе санауышқа қатар өзгеріс енгізу жарысу жағдайын тудыра алады. count = count + 1 мағыналық тұрғыдан оқу, есептеу және жазудан тұрады; бірнеше ағын осы қадамдарды араластыра алады. Құрама әрекетті Lock қорғайды. Бір операцияны кездейсоқ атомарлы деп қабылдау бағдарлама келісімін алмастырмайды.

Әдеттегі GIL қосылған CPython жинағында бірнеше ағын таза Python есептеуін барлық өзекке қатар таратпайды. Желілік енгізу-шығару кезінде ағындар күту уақытын қабаттастыра алады. CPU жұмысы көп есепке процестер пулы қарастырылады. GIL-сіз жинақтарда орындау қасиеті өзгереді, бірақ ортақ күйді синхрондау талабы сақталады. [11]

Процестер жадты тікелей ортақ пайдаланбайды; ақпарат алмасу сериализация мен бөлек байланыс арнасын талап етеді. Бұл шығын мен оқшаулауды бірге береді. Таңдау «ағын әрқашан жақсы» деген ережемен емес, жұмыс сипаты, күй көлемі және ақауды оқшаулау талабымен жасалады.

Шектелген кезегі бар сервер

Келесі сервер 5 дәрістегі protocol.py модулін пайдаланады. Төрт жұмысшы бір мезгілде төрт қосылымды өңдейді. Кезекте сегіз қосылымға орын бар. Әр қосылым бір сұрау мен бір жауаптан кейін жабылады. Кезек толса, сервер жаңа қосылымды жабады; толық қызметте бұл жағдай клиентке келісілген бос емес мәртебесімен хабарланады.

week06_server.py

```
import queue
import socket
import threading
from protocol import recv_message, send_message

jobs = queue.Queue(maxsize=8)

def worker():
    while True:
```

```
        conn = jobs.get()
        try:
            with conn:
                conn.settimeout(5)
                request = recv_message(conn)
                if request.get("op") != "echo":
                    reply = {"ok": False, "error": "unknown_op"}
                else:
                    reply = {"ok": True, "text": request.get("text")}
                send_message(conn, reply)
        except (OSError, ValueError, EOFError, RecursionError) as error:
            print(type(error).__name__)
        finally:
            jobs.task_done()

    for _ in range(4):
        threading.Thread(target=worker, daemon=True).start()

    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
        server.bind(("127.0.0.1", 9002))
        server.listen(16)
        print("Worker server on 127.0.0.1:9002")
        while True:
            conn, address = server.accept()
            try:
                jobs.put_nowait(conn)
            except queue.Full:
                conn.close()
```

week06_client.py

```
    import socket
    from protocol import recv_message, send_message

    with socket.create_connection(("127.0.0.1", 9002), timeout=5) as sock:
        send_message(sock, {"op": "echo", "text": "Қатар өңдеу"})
        print(recv_message(sock))
```

Іске қосу: python week06_server.py және басқа терминалдарда python week06_client.py. Күтілетін жауДәріс. ok мәні True және жіберілген мәтін бар. queue.Queue ағындар арасында қауіпсіз тапсыруды қамтамасыз етеді; task_done тапсырманың кезектен алынғаннан кейін аяқталғанын белгілейді. queue.join тек task_done саны алынған тапсырмалар санымен теңескенде қайтады. [12]

Бұғаттау тәртібі және өмірлік цикл

Lock тек ортақ күйді өзгертетін қысқа аймақты қорғауы тиіс. Желіден жауап күтіп тұрып ортақ құлыпты ұстау басқа ағындарды тоқтатады. Мысалы, барлық чат клиенттеріне хабар жібererде алдымен құлып ішінде клиенттер тізімінің көшірмесін алыңыз, кейін құлыптан тыс жіберіңіз. Бұл кезде жабылған сокетке жіберу қатесін жеке өңдеу қажет. Бірнеше құлып қолданылса, барлық ағын бірдей алу ретін ұстанады; әйтпесе өзара күту пайда болады.

Мысалдағы daemon ағындар негізгі процесс тоқтағанда бірге аяқталады. Бұл сабақ кодын қысқартады, бірақ аяқталмаған сұрауларды сақтап қалмайды. Реттелген тоқтату үшін жаңа қосылым қабылдауды тоқтату, кезекті аяқтау, әр жұмысшыға аяқтау белгісін жіберу және join арқылы ағындарды күту тәртібі керек. Кезекте өткізілген уақыт та жалпы сұрау мерзіміне кіруі тиіс; тек жұмысшының recv timeout мәні кезекте тұруды шектемейді.

Жүктеме туралы есеп

Төрт жұмысшының әр сұрауға орташа 0,2 секунд жұмсайтын жағдайында идеал қызмет шегі секундына 20 сұрау: әр жұмысшы секундына 5 сұрау орындайды. Бұл есеп желі, құлып және жоспарлау шығынын қоспайды. Кіріс қарқыны ұзақ уақыт секундына 30 сұрау болса, кезек тұрақты өседі. Кезек көлемін арттыру қызмет қабілетін арттырмайды; ол тек артық жүктеменің көрінуін кешіктіреді.

Бақылау сұрақтары

1. Қатар және параллель орындау қалай ажыратылады?
2. Неліктен шексіз кезек қауіпті?
3. Құлып ішінде recv шақыру қандай әсер береді?
4. daemon ағындар мен реттелген тоқтатудың айырмасы қандай?

Өздік жұмыс: ортақ өңделген сұраулар санауышын қосып, оны Lock арқылы қорғаңыз. Жұмысшылар санын 1 және 4 етіп, бірнеше клиентпен салыстырыңыз. Кестеге аяқталған сұрау саны, қате саны және жалпы уақытты жазыңыз. Қорытындыда кезек көлемінің өткізу қабілеті мен күтуге әсерін ажыратыңыз.

Дәрісті өткізу реті: архитектураға 20 минут, ағын мен күйге 20 минут, серверге 30 минут, жүктеме есебіне 10 минут, бекітуге 10 минут.

7 Дәріс. Асинхронды желілік бағдарламалау

Дәріс мақсаты: оқиғалар циклі мен корутиналарға сүйеніп, бірнеше қосылымды бір орындау ағынында басқару.

Оқу нәтижелері: студент blocking және nonblocking айырмасын түсіндіреді, async және await қолданады, asyncio ағындарымен жұмыс істейді және баяу клиентке қатысты ресурстарды шектейді.

Күту мен оқиғалар циклі

Бұғаттайтын recv дерек келгенше ағымдағы орындауды тоқтатады. Бір ағын жүз қосылымды кезекпен осылай оқыса, алғашқы үнсіз клиент қалғандарының өңделуін кешіктіреді. Бұғаттамайтын сокет дайын дерек жоқ кезде басқаруды қайтарады. Бағдарлама операциялық жүйеден қай сокеттің оқуға немесе жазуға дайын екенін сұрап, тек дайын нүктелермен жұмыс істей алады. select, poll және басқа платформалық тетіктер осы модельге негіз береді.

asyncio төменгі деңгейдегі дайындық хабарламаларын корутина түріндегі кодпен байланыстырады. async def функциясын шақыру корутина нысанын жасайды; оның орындалуын await, create_task немесе asyncio.run ұйымдастырады. await орындалатын әрекет күтуге көшкенде оқиғалар циклі басқа дайын міндетке басқару береді. Бұл ынтымақты жоспарлау: ұзақ үздіксіз есептеу басқаруды қайтармаса, басқа міндеттер де кешігеді.

Асинхронды код өздігінен жылдам есептеу қозғалтқышына айналмайды. Ол енгізу-шығару күтуін тиімді қабаттастырады. time.sleep бүкіл оқиғалар циклін тоқтатады, ал await asyncio.sleep басқа міндеттерге уақыт береді. Бұғаттайтын файлдық не кітапханалық әрекетті asyncio.to_thread арқылы бөлек ағынға шығару орынды; ауыр CPU есебі үшін процестер пулы қарастырылады.

Ағындық интерфейс және кері қысым

asyncio.start_server әр қабылданған қосылымға StreamReader және StreamWriter береді. reader.readexactly(n) дәл n байтты күтеді; қосылым ерте аяқталса IncompleteReadError көтереді. writer.write деректі шығыс буферіне қосады. await writer.drain буфердің тасуын шектеуге қатысады, бірақ қашықтағы қолданба деректі өңдегенін растамайды. [8]

Кері қысым — қабылдаушы немесе төменгі буфер үлгермегенде жоғарыдағы өндірушіні баяулату. drain шақырмай шексіз write жасау жадтың өсуіне әкелуі мүмкін. Кері қысымды тек сокетке сүйеу жеткіліксіз: қолданбалық хабар кезегі, белсенді сұрау саны және хабар көлемі де шектеледі. Баяу тұтынушы үшін кезекті шектеу, ескі жаңартуды алмастыру немесе қосылымды тоқтату саясаты керек.

Бір жолдық хаттамасы бар сервер

Бұл мысалда әр сұрау UTF-8 мәтіні мен жаңа жол таңбасынан тұрады. Бір клиент бірнеше жол жібере алады. Бір жолдың байт көлемі жаңа жолмен бірге 1024-тен аспайды. Сервер жолға жауап береді. Оқу мерзімі 10 секунд, жазу мерзімі 5 секунд. 5 дәрістегі ұзындық префиксінің орнына мұнда жаңа жолмен шекаралау саналы түрде таңдалған.

week07_server.py

```
import asyncio

MAX_LINE = 1024

async def handle(reader, writer):
    try:
        while True:
```

```
            async with asyncio.timeout(10):
                data = await reader.readline()
            if not data:
                break
            if len(data) > MAX_LINE or not data.endswith(b"\n"):
                raise ValueError("Invalid line")
            text = data[:-1].decode("utf-8")
            reply = ("OK " + text + "\n").encode("utf-8")
            writer.write(reply)
            async with asyncio.timeout(5):
                await writer.drain()
    except (TimeoutError, OSError, ValueError) as error:
        print(type(error).__name__)
    finally:
        writer.close()
        try:
            await writer.wait_closed()
        except OSError:
            pass

async def main():
    server = await asyncio.start_server(
        handle, "127.0.0.1", 9003, limit=MAX_LINE
    )
    async with server:
        await server.serve_forever()

if __name__ == "__main__":
    asyncio.run(main())
```

week07_client.py

```
import asyncio

async def main():
    async with asyncio.timeout(5):
        reader, writer = await asyncio.open_connection(
            "127.0.0.1", 9003
        )
        try:
            writer.write("Асинхронды сұрау\n".encode("utf-8"))
            await writer.drain()
            print((await reader.readline()).decode("utf-8").strip())
        finally:
            writer.close()
            await writer.wait_closed()

if __name__ == "__main__":
    asyncio.run(main())
```

Іске қосу: python week07_server.py және python week07_client.py. Күтілетін жол: «OK Асинхронды сұрау». Серверде limit кіріс буфері мен жолды оқу шегін басқаруға қатысады, ал len тексеруі қолданба келісімін нақты бекітеді. Ол барлық клиенттердің жалпы жадын шектемейді; қосылым санына бөлек саясат қажет.

Міндеттер өмірлік циклі және тоқтату

create_task фондық міндетті жоспарлайды. Оның сілтемесін сақтамай, нәтижесін еш жерде күтпей тастау қателерді бақылауды қиындатады. Байланысты міндеттерді TaskGroup ішінде басқару олардың өмірлік циклін

бір жерге жинайды. Топтағы міндеттердің бірі қате берсе, қалғандарына бас тарту таралады; ресурстар finally ішінде жабылуы керек.

Бас тарту міндеттің кез келген күту нүктесінде келуі мүмкін. CancelledError-ды негізсіз жұтып қою сыртқы уақыт шектеуін немесе қызметті тоқтатуды бұзады. Тазалау әрекеттерін орындағаннан кейін бас тартудың жоғарыға өтуін сақтау қажет. Бір сұраудың мерзімі бүкіл оқу, тексеру және жазуды қамтысын ба, әлде әр кезең бөлек шектелсін бе — бұл қызмет келісімінде анықталады.

Асинхронды архитектураны бағалау

Асинхронды модель көп күтуі бар қосылымдарда ыңғайлы, бірақ қате құлыптау мен шексіз міндет жасау мәселесін жоймайды. Semaphore бір мезгілдегі белсенді әрекеттерді шектей алады. Алайда семафорды күтетін міндеттер саны шексіз қалса, жад мәселесі сақталады. Жүйенің қабылдау нүктесінде жүктемені шектеу мен өлшемдерді бақылау қажет.

Бақылау сұрақтары

1. await басқаруды әр шақырылғанда міндетті түрде бере ме?
2. time.sleep оқиғалар цикліне қалай әсер етеді?
3. drain нені шешеді және нені растамайды?
4. Неге белсенді міндеттер мен күтетін кезек бөлек шектеледі?

Өздік жұмыс: клиентте TaskGroup арқылы 20 қысқа сұрауды қатар орындаңыз. Әр сұраудың нәтижесі мен уақытын тіркеңіз. Сервер өңдеуіне await asyncio.sleep(0.2) қосып, кейін оны time.sleep(0.2) нұсқасымен салыстырыңыз. Екінші нұсқаның әсерін оқиғалар циклінің жұмысымен түсіндіріңіз; тәжірибеден кейін бұғаттайтын нұсқаны алып тастаңыз.

Дәрісті өткізу реті: оқиғалар цикліне 25 минут, ағындық API-ға 20 минут, кодқа 25 минут, міндеттер өміріне 10 минут, бекітуге 10 минут.

8 Дәріс. HTTP хаттамасы және веб клиент

Дәріс мақсаты: HTTP сұрауы мен жауабының құрылымын түсініп, жауапты шектеп оқитын веб клиент жазу.

Оқу нәтижелері: студент әдіс, мәртебе, тақырып және денені ажыратады; URL параметрлерін дұрыс кодтайды; желілік қатені HTTP қатесінен бөледі.

HTTP алмасуының мағынасы

HTTP ресурстармен әрекеттесу мағынасын анықтайды. Клиент сұрауда әдісті, ресурс нысанасын және қосымша тақырыптарды жібереді. Сервер мәртебе кодын, тақырыптарды және қажет болса денені қайтарады. HTTP/1.1 ішінде бастапқы жол мен тақырыптар мәтіндік пішімде беріледі; бос жол тақырып пен денені ажыратады. HTTP/2 және HTTP/3 осы мағыналарды басқа кадрлау тәсілдерімен тасымалдайды.

GET ресурстың көрінісін алады, HEAD GET жауабының денесіз метадерегін сұрайды, POST ресурсқа өңдеу үшін дерек ұсынады, PUT нысана ресурстың күйін ауыстыруға арналған. Қауіпсіз әдіс клиент сұраған мағына бойынша сервер күйін өзгертуді талап етпейді. Идемпотентті әдісті бірдей сұраумен қайталау нысанадағы көзделген әсерді қайталап ұлғайтпайды; жауап кодының әр жолы бірдей шығуы міндет емес. [5]

Идемпотенттілік пен қауіпсіздік сөздерінің HTTP ішіндегі мағынасы ақпараттық қауіпсіздікпен бірдей емес. GET сұрауы аутентификация талап етуі және құпия дерек қайтаруы мүмкін. DELETE идемпотентті мағынаға ие, бірақ ресурсты жояды. POST көбіне қайталауға қосымша идентификатор талап етеді. HTTP әдісінің атауы қолданбадағы қате жобаланған операцияны автоматты түрде түземейді.

Мәртебе кодтары және клиент шешімі

Кодтар тобы	Мағынасы	Клиент әрекетінің үлгісі
2xx	Сұрау сәтті өңделген	Денені келісім бойынша оқу
3xx	Қосымша бағыттау не кэш әрекеті	Location пен кэш шарттарын тексеру
4xx	Сұрауға не қатынауға қатысты мәселе	Параметрді не рұқсатты түзету
5xx	Сервер жағындағы мәселе	Әдіс пен саясатқа сай шектеулі қайталау

200 — қалыпты сәтті жауап, 201 — ресурс жасалғанын, 204 — жауап денесінің жоқтығын көрсетеді. 400 қате сұрауға, 401 аутентификацияға, 403 рұқсаттың жоқтығына, 404 ресурс табылмауына қолданылады. 429 қарқын шегін көрсетуге пайдаланылады. Клиент барлық 4xx кодын қайта жібермеуі тиіс: қате параметр қайталанған сайын түзелмейді. 503 кезінде де шексіз қайталау қызметті қалпына келтіруге кедергі келтіреді.

Тақырыптар және дене көлемі

Content-Type дененің медиа пішімін көрсетеді. Content-Length таңба санын емес, дене байттарының санын береді. Accept клиент қабылдайтын пішімді көрсетеді. Authorization қатынау дерегін тасиды; оны журналға ашық жазуға болмайды. Cache-Control кэш мінезін басқарады. HTTP тақырып атаулары әріп регистріне тәуелсіз оқылады.

HTTP/1.1 ішінде дене ұзындығын анықтау Content-Length, Transfer-Encoding және жауаптың мағыналық ережелеріне байланысты. Клиент кітапханасын пайдалану осы күрделі ережелерді қолмен қайта жазуды азайтады. Chunked тасымалдау кодтауы мен JSON ішіндегі қолданбалық бөліктерді шатастырмаңыз. Қысылған жауаптың желідегі көлемі ашылғаннан кейінгі көлемнен аз болуы мүмкін; екі кезеңге де шек қажет.

Стандартты кітапханамен HTTP клиент

Алдымен бөлек терминалда python -m http.server 8000 --bind 127.0.0.1 командасын орындаңыз. Бұл сервер ағымдағы каталогты ұсынады. Тәжірибеге тек оқу файлдары бар бөлек каталог таңдаңыз. Одан кейін келесі клиентті іске қосыңыз. http.server оқу мен жергілікті тексеруге арналған; оны қызметті жариялау шешімі ретінде қолданбаңыз. [9]

week08_client.py

```
from urllib.error import HTTPError, URLError
from urllib.request import Request, urlopen

MAX_BODY = 100_000
request = Request(
    "http://127.0.0.1:8000/",
    headers={"Accept": "text/html", "User-Agent": "NetworkCourse/1.0"},
    method="GET",
)
try:
    with urlopen(request, timeout=5) as response:
        data = response.read(MAX_BODY + 1)
        if len(data) > MAX_BODY:
            raise ValueError("Response exceeds limit")
        print("status:", response.status)
        print("content-type:", response.headers.get("Content-Type"))
        print("bytes:", len(data))
except HTTPError as error:
    print("HTTP error:", error.code)
    error.close()
except URLError as error:
    print("Connection error:", error.reason)
except (TimeoutError, ValueError, OSError) as error:
    print(type(error).__name__, str(error))
```

Іске қосу: python week08_client.py. Күтілетін мәртебе — 200, пішім — text/html, ал байт саны каталог құрамына тәуелді. URL жолын /missing-resource-123 мәніне өзгерткенде әдетте 404 шығады. Серверді тоқтатып қайталағанда HTTP мәртебесі емес, байланыс қатесі алынады. urllib.request HTTPError-ды URLError-дан бұрын ұстау керек, себебі HTTPError сол иерархияға кіреді. [10]

URL кодтауы және қайта бағыттау

URL параметрін жолдарды тікелей қосып құрастыру & немесе = таңбалары бар мәнді бұзады. urlencode кілттер мен мәндерді URL ережесіне сай кодтайды. JSON денесін quote арқылы өңдеуге болмайды: URL кодтауы мен дене сериализациясы бөлек қабаттар. Құпия сөзді query ішіне енгізу тарих, прокси және журнал арқылы таралу қаупін өсіреді.

Клиент кітапханасы қайта бағыттауды автоматты орындауы мүмкін. Пайдаланушы берген кез келген URL-ға серверден сұрау жіберу ішкі мекенжайларға қатынау қаупін туғызады. Қызмет тек рұқсат етілген нысаналарды қолданса, схема, хост, порт және әр қайта бағытталған нысана тексеріледі. Бұл курс клиенті бекітілген жергілікті URL-ды пайдаланады.

Бақылау сұрақтары

1. Content-Length неге len(text) мәніне тең болмауы мүмкін?
2. 404 пен ConnectionRefusedError айырмасы қандай?
3. Идемпотентті әрекет міндетті түрде қауіпсіз әдіс пе?

4. Жауап көлемін неге шектеу қажет?

Өздік жұмыс: клиенттен мәртебе, Content-Type және жауап уақытын кестеге жазыңыз. Бар ресурс, жоқ ресурс және тоқтаған сервер жағдайларын салыстырыңыз. URL параметрі ретінде «желілік бағдарламалау» мәтінін urlencode арқылы кодтап, бос орын мен қазақ әріптерінің өзгеруін түсіндіріңіз.

Дәрісті өткізу реті: HTTP құрылымына 25 минут, әдістер мен мәртебеге 20 минут, клиентке 25 минут, URL мәселесіне 10 минут, бекітуге 10 минут.

9 Дәріс. REST қағидалары және JSON веб қызметі

Дәріс мақсаты: ресурстарға негізделген шағын HTTP API құрып, кіріс деректі тексеру мен жауап келісімін ұйымдастыру.

Оқу нәтижелері: студент ресурс пен операцияны ажыратады, HTTP әдістерін қолданады, JSON денесін тексереді және API келісімін жазады.

Ресурсқа негізделген интерфейс

API — бағдарламалардың өзара әрекеттесу келісімі. Ресурсқа негізделген жобалауда URL затты не жинақты көрсетеді: /notes жазбалар жинағын, /notes/17 жеке жазбаны білдіреді. Әрекет HTTP әдісімен анықталады. Осындай тұрақты атаулар клиенттің сервердің ішкі функция атауларына тәуелділігін азайтады.

REST архитектурасы клиент пен сервер міндетін бөлу, сұраудың өзін жеткілікті сипаттауы, кэштеу, біркелкі интерфейс және қабатты жүйе қағидаларын біріктіреді. Әр JSON API осы талаптардың бәрін орындамайды. Курста ресурстық атаулар, HTTP мағынасы және сұраудың айқын келісімі қолданылады. Сеанс күйін серверде жасырын жинау клиент сұрауының дербестігін әлсіретеді; ал дереккордағы ресурс күйін сақтау REST қағидасына қайшы емес. [22]

Коллекция үлкен болса, оны бір жауапқа түгел жинау жад пен желі шығынын арттырады. Беттеу көлем шегін, тұрақты сұрыптау ретін және келесі бет белгісін талап етеді. offset үлгісі түсінікті, бірақ дерек өзгергенде элементтер қайталануы немесе өткізіліп кетуі мүмкін. Курсорға негізделген беттеу үлкен өзгермелі жинақта ыңғайлырақ; курсордың жарамдылық ережесі де құжатталады.

API келісімін жазу

Жақсы келісім URL мен әдісті ғана атамайды. Ол сұрау өрістерін, дерек түрлерін, рұқсат етілген ұзындықты, жауап мәртебесін, қате кодын және қайталау тәртібін көрсетеді. Пайдаланушыға арналған хабарды машиналық error мәнінен бөлу клиент логикасын тұрақтандырады. Нұсқаға қатысты өзгерістер алдын ала жоспарланады.

Әдіс және жол	Қызметі	Сәтті жауап	Қате үлгісі
GET /health	Қызмет процесін тексеру	200 және status	Байланыс қатесі
GET /notes	Жадтағы жазбаларды оқу	200 және массив	500 ішкі ақау
POST /notes	Қысқа жазба қосу	201 және жасалған жазба	400, 411, 413, 415

Төмендегі сервер HTTP/1.0 жауаптарын пайдаланып, әр сұраудан кейін қосылымды жабады. Бұл оқу мысалында тұрақты қосылым, chunked дене, аутентификация және тұрақты дереккор жоқ. POST денесі Content-Length арқылы беріледі; басқа тасымалдау кодтауы қабылданбайды. Жазбалар саны 1000-мен, мәтін ұзындығы 200 таңбамен шектеледі.

Жергілікті JSON сервер

week09_api.py

```
import json
from http.server import BaseHTTPRequestHandler, HTTPServer

notes = []

class Handler(BaseHTTPRequestHandler):
    def setup(self):
        super().setup()
        self.connection.settimeout(5)
```

```
    def reply(self, code, payload):
        body = json.dumps(payload, ensure_ascii=False).encode("utf-8")
        self.send_response(code)
        self.send_header("Content-Type", "application/json; charset=utf-8")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def do_GET(self):
        if self.path == "/health":
            self.reply(200, {"status": "ok"})
        elif self.path == "/notes":
            self.reply(200, notes)
        else:
            self.reply(404, {"error": "not_found"})

    def do_POST(self):
        if self.path != "/notes":
            self.reply(404, {"error": "not_found"})
            return
        if self.headers.get("Transfer-Encoding") is not None:
            self.reply(400, {"error": "unsupported_transfer"})
            return
        if self.headers.get_content_type() != "application/json":
            self.reply(415, {"error": "json_required"})
            return
        lengths = self.headers.get_all("Content-Length", [])
        if len(lengths) != 1 or not lengths[0].isdigit():
            self.reply(411, {"error": "one_length_required"})
            return
        size = int(lengths[0])
        if not 0 < size <= 4096:
            self.reply(413, {"error": "body_limit"})
            return
        try:
            raw = self.rfile.read(size)
            if len(raw) != size:
                raise ValueError("Incomplete body")
            obj = json.loads(raw.decode("utf-8"))
            text = obj.get("text") if isinstance(obj, dict) else None
            if not isinstance(text, str) or not 1 <= len(text.strip()) <= 200:
                raise ValueError("Invalid text")
        except (ValueError, OSError, RecursionError):
            self.reply(400, {"error": "invalid_body"})
            return
        if len(notes) >= 1000:
            self.reply(503, {"error": "capacity_reached"})
            return
        note = {"id": len(notes) + 1, "text": text.strip()}
        notes.append(note)
        self.reply(201, note)

if __name__ == "__main__":
    with HTTPServer(("127.0.0.1", 8001), Handler) as server:
        print("API on http://127.0.0.1:8001")
        server.serve_forever()
```

API клиенті және нәтижені талдау

week09_client.py

```
import json
from urllib.request import Request, urlopen
```

```
url = "http://127.0.0.1:8001/notes"
body = json.dumps({"text": "Алғашқы жазба"}).encode("utf-8")
request = Request(
    url, data=body, method="POST",
    headers={"Content-Type": "application/json"},
)
with urlopen(request, timeout=5) as response:
    print(response.status, json.load(response))
with urlopen(url, timeout=5) as response:
    print(response.status, json.load(response))
```

Іске қосу: python week09_api.py, кейін python week09_client.py. Алғашқы жауап 201 және id мен text өрістері бар нысан береді. Екінші жауап 200 және жазбалар тізімін береді. Сервер қайта іске қосылса, тізім босайды: күй тек жадта сақталады. Бұл мінез тұрақты сақтау кепілдігімен шатастырылмауы тиіс.

HTTPServer бір сұрауды бір уақытта өңдейді. Оны ThreadingHTTPServer-ге ауыстырғанда notes тізіміне байланысты «идентификатор жасау және қосу» әрекеті құлыппен қорғалуы қажет. Дерекқорға көшкенде бірегейлік пен транзакция ережесі дерекқор деңгейінде бекітіледі. Жадтағы тізімді дерекқордың баламасы деп қарау қатар орындалу мен қалпына келтіру мәселелерін жасырады.

Валидация және жауап сапасы

Валидация үш деңгейде жүреді: тасымал денесінің көлемі, синтаксистік пішім және өрістің мағынасы. Дұрыс JSON ішіндегі теріс баға немесе бос атау мағыналық тұрғыдан жарамсыз болуы мүмкін. Пайдаланушы жіберген id арқылы басқа адамның жазбасын өзгертуге жол бермеу авторизацияны талап етеді; тек JSON тексеруі мұны шешпейді. Қате жауабында ішкі файл жолын, құпияны немесе трассаны жарияламаңыз.

Бақылау сұрақтары

1. Ресурс пен функция атауы қалай ажыратылады?

2. Неге сервер қайта қосылғанда жазбалар жоғалады?

3. JSON синтаксисі мен өріс валидациясының айырмасы қандай?

4. Ағындық серверге ауыстырғанда қандай ортақ әрекет қорғалуы тиіс?

Өздік жұмыс: GET /notes/ID әрекетін қосыңыз. Бар идентификаторға 200, жоқ идентификаторға 404 қайтарыңыз. Бос мәтін, 201 таңбалық мәтін және қате Content-Type үшін сынақ кестесін құрыңыз. Қосымша жобалау тапсырмасы: беттеуге арналған limit параметрінің әдепкі және ең үлкен мәнін анықтаңыз.

Дәрісті өткізу реті: ресурс пен келісімге 25 минут, сервер талдауына 30 минут, клиентке 15 минут, валидацияға 10 минут, бекітуге 10 минут.

10 Дәріс. Нақты уақыттағы байланыс WebSocket және SSE

Дәріс мақсаты: сервердегі өзгерісті клиентке жедел жеткізу тәсілдерін салыстырып, оқиғалар ағынын құрастыру.

Оқу нәтижелері: студент polling, long polling, SSE және WebSocket арасынан талапқа сай тәсіл таңдайды; оқиға шекарасы, қайта қосылу және баяу клиент мәселесін түсіндіреді.

Жаңартуды жеткізу тәсілдері

Қарапайым polling кезінде клиент белгілі аралықпен серверден жаңа күй сұрайды. Бұл әдіс іске асыруға оңай, бірақ өзгеріс жоқ кезде де сұрау шығынын тудырады. Интервал қысқарса жаңарту кешігуі азаяды, сұрау саны өседі. Long polling кезінде сервер сұрауды жаңарту пайда болғанша немесе мерзімі біткенше ұстайды. Жауаптан кейін клиент келесі сұрауды бастайды.

Мысал: 500 клиент әр 2 секунд сайын жаңарту сұраса, серверге секундына орташа 250 сұрау түседі. Өзгеріс минутына бір рет қана орын алса, сұраулардың басым бөлігі жаңа ақпарат әкелмейді. Оқиғаға сүйенген арна бос сұрауларды азайтады, бірақ ұзақ қосылым, прокси мерзімі, қайта қосылу және ресурсты басқару мәселелерін енгізеді. Тәсіл таңдау тек сұрау санына емес, екіжақты алмасу қажеттілігіне де сүйенеді.

Тәсіл	Дерек бағыты	Клиент әрекеті	Негізгі қолданылуы
Polling	Сұрау және жауап	Мерзімді сұрайды	Сирек жаңаратын күй
Long polling	Сұрау және кеш жауап	Әр жауаптан кейін қайта сұрайды	Оқиғаны күтетін API
SSE	Серверден клиентке ағын	EventSource арқылы тыңдайды	Хабарландыру, көрсеткіштер
WebSocket	Екі бағытта	Ашық арнада хабар алмасады	Чат, бірлескен өңдеу

WebSocket қызметі

Классикалық WebSocket қосылымы HTTP/1.1 Upgrade сұрауымен басталып, сервер 101 мәртебесін қайтарғаннан кейін кадрлық алмасуға көшеді. Кадр мәтіндік немесе бинарлық дерек таси алады; ping, pong және close басқару кадрлары бар. Клиенттен серверге жіберілген кадрлар маскаланады, бірақ маскалау шифрлау қызметін атқармайды. Қорғалған арна wss схемасы арқылы TLS пайдаланады. [14]

WebSocket ұзақ арна береді, бірақ қолданбалық хабардың мағынасын анықтамайды. Чатқа арналған хабарда kind, room, id және text өрістері қажет болуы мүмкін. Рұқсат әр бөлмеге жеке тексеріледі. Бір қосылымның аутентификациядан өтуі барлық ресурсқа қатынау бермейді. Origin тексеруі браузерлік клиентке қатысты пайдалы, бірақ пайдаланушыны аутентификациялаудың орнына жүрмейді.

Клиент арнаның жабылғанын анықтап, үзіліс аралығын ұлғайтып қайта қосылады. Соңғы алынған хабар идентификаторы үзілісте жоғалған оқиғаларды сұрауға көмектеседі. Сервердің сақтау тарихы жеткіліксіз болса, клиент толық күйді қайта алады. Бір хабарды екі рет қабылдау ықтималдығына қарсы id бойынша қайталануды сүзу енгізіледі. Қосылымның бар болуы дерек жоғалмайтынын жеке өзі кепілдемейді.

SSE оқиғасының пішімі

SSE HTTP арқылы серверден браузерге UTF-8 мәтіндік оқиғалар жібереді. Жауаптың Content-Type мәні text/event-stream. data жолдары оқиға денесін, id идентификаторды, event оқиға атауын көрсете алады; бос жол оқиғаны аяқтайды. Браузер EventSource көмегімен ағынды қабылдап, үзілгенде қайта қосыла алады. Last-Event-ID қайта жалғауда серверге соңғы оқиғаны көрсетуге қолданылады. [20]

SSE бір бағыттағы оқиғаларға қолайлы. Клиент серверге пәрмен жіберуі керек болса, оны бөлек HTTP сұрауымен орындай алады. Екі бағыттағы жиі хабар алмасуға WebSocket ыңғайлырақ. Аралық прокси жауапты буферлесе, сервер flush жасағанның өзінде клиент жаңартуды кеш алады; орналастыруда прокси баптауы тексеріледі.

Браузерде оқиға қабылдау мысалы

Бағдарлама бір портта HTML бетін және /events ағынын ұсынады. Сервер он оқиға жібереді. Браузер соңғысын алғаннан кейін арнаны жабады. Қайта қосылған клиентке сервер жаңа санақ бастайды; бұл мысалда оқиға тарихын қалпына келтіру жоқ.

week10_sse.py

```
import time
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

PAGE = b"<!doctype html><meta charset="utf-8">
<title>SSE demo</title><h1>Server events</h1><pre id="out"></pre>
<script>
const output = document.getElementById("out");
const source = new EventSource("/events");
source.onmessage = (event) => {
  output.textContent += event.data + "\\n";
  if (event.lastEventId === "10") source.close();
};
</script>"

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        self.connection.settimeout(5)
        if self.path == "/":
            self.send_response(200)
            self.send_header("Content-Type", "text/html; charset=utf-8")
            self.send_header("Content-Length", str(len(PAGE)))
            self.end_headers()
            self.wfile.write(PAGE)
        elif self.path == "/events":
            self.send_response(200)
            self.send_header("Content-Type", "text/event-stream")
            self.send_header("Cache-Control", "no-cache")
            self.end_headers()
            try:
                for number in range(1, 11):
                    text = f"id: {number}\ndata: event {number}\n\n"
                    self.wfile.write(text.encode("utf-8"))
                    self.wfile.flush()
                    time.sleep(1)
            except OSError:
                pass
            self.close_connection = True
        else:
            self.send_error(404)

if __name__ == "__main__":
    with ThreadingHTTPServer(("127.0.0.1", 8002), Handler) as server:
        print("Open http://127.0.0.1:8002/")
        server.serve_forever()
```

Іске қосу: python week10_sse.py. Браузерде http://127.0.0.1:8002/ адресін ашыңыз. event 1 жолынан event 10 жолына дейінгі хабарлар бір секунд аралықпен көрінеді. textContent мәтінді HTML ретінде орындамай көрсетеді. Бұл ағындық серверде time.sleep жеке жұмысшы ағынды күттіреді; оны 7 дәрістегі оқиғалар цикліне көшіру үшін asyncio.sleep керек.

Арнаны пайдалану шектеулері

Жеке ағынды әр SSE клиентіне беру ұзақ қосылымдар көбейгенде қымбаттайды. Нақты қызметте асинхронды сервер, қосылым санының шегі, heartbeat және бос тұрған арнаның мерзімі қарастырылады. Heartbeat байланыстың тірі екенін бақылауға көмектеседі; ол соңғы бизнес оқиғасының клиентте сақталғанын растамайды.

Баяу клиентке арналған кезек толғанда барлық серверді тоқтатпау керек. Күй көрсеткіштері үшін ескі өлшемді жаңасымен алмастыруға болады. Ал төлем оқиғасын тастауға болмайды: тұрақты журнал мен қайта оқу қажет. «Оқиға» мен «соңғы күй» арасындағы айырма арна саясатын анықтайды.

Бақылау сұрақтары

1. Polling аралығы сұрау санына қалай әсер етеді?
2. WebSocket маскалауы неге шифрлау емес?
3. SSE оқиғасын қандай белгі аяқтайды?
4. Қайта қосылған клиент қандай жағдайда толық күйді қайта сұрайды?

Өздік жұмыс: SSE хабарын сервер уақыты мен оқиға нөмірі бар JSON мәтініне ауыстырыңыз. Екі браузер бетін ашып, олардың тәуелсіз қосылғанын тексеріңіз. Бетті бесінші оқиғадан кейін жапқанда сервердің жұмысын жалғастыратынын бақылаңыз. Чат пен біржақты бағалар тақтасына арна таңдауыңызды негіздеңіз.

Дәрісті өткізу реті: тәсілдерді салыстыруға 20 минут, WebSocket-ке 20 минут, SSE-ге 20 минут, мысалға 20 минут, бекітуге 10 минут.

11 Дәріс. TLS және желілік қолданбаның қауіпсіздігі

Дәріс мақсаты: қорғалған арнаны дұрыс құрып, серверді тексеру, қолданушыны тану және рұқсат беру міндеттерін ажырату.

Оқу нәтижелері: студент TLS кепілдіктерін түсіндіреді, сертификат пен хост атауын тексереді, жергілікті TLS клиенті мен серверін іске қосады және құпияларды өңдеу қағидаларын қолданады.

Қорғаудың бірнеше міндеті

Құпиялық деректі бөгде тараптың оқуынан қорғайды. Тұтастық жолдағы өзгерісті анықтауға көмектеседі. Аутентификация қатысушының кім екенін тексереді. Авторизация сол қатысушының қандай әрекетке құқығы барын анықтайды. TLS арнаны қорғайды, бірақ қолданбадағы рұқсат ережесін өзі құрмайды. Сертификаты дұрыс сервердің API-ында басқа адамның жазбасын өзгертуге рұқсат беретін қате сақталуы мүмкін.

TLS келісуі криптографиялық параметрлерді келісіп, кілт материалын құрып, сервердің түпнұсқалығын тексеруге мүмкіндік береді. TLS 1.3 бұрынғы нұсқаларға қарағанда келісім мен шифр жинақтарын жаңартады. Ерте дерек жіберу режиміндегі қайталап ойнату тәуекелі күй өзгертетін операцияларға ерекше талап қояды. Сондықтан 0-RTT мүмкіндігін қолдану жеке қолданбалық талдауды талап етеді. [15]

Қорғалған арна үшін сертификат тізбегін тексеру жеткіліксіз: сертификат клиент күткен хостқа сәйкес келуі тиіс. Пайдаланушы example.org адресіне қосылса, бөтен атауға берілген жарамды сертификат қабылданбайды. Сенімді орталық, жарамдылық мерзімі және атау сәйкестігі бірге қаралады. Жүйелік уақыттың қате болуы мерзімді тексеруге әсер етеді.

Python ішіндегі TLS контексті

ssl.create_default_context клиентке арналған сертификат тексеруін қосатын баптаулар береді. wrap_socket ішіндегі server_hostname атауды тексеру мен SNI үшін қолданылады. CERT_NONE орнату немесе check_hostname өшіру қосылым қатесін жасырып, қарсы тарапты тексеруден бас тартады. TLS қатесін түзеткенде сенім тізбегі, атау және уақыт себептері зерттеледі. [17]

Оқу тәжірибесінде localhost үшін қысқа мерзімді өз қолымен қол қойылған сертификат жасалады. Клиент оған тек cafile параметрі арқылы нақты сенеді. Бұл тәсіл жүйенің барлық сертификат тексеруін өшірмейді. Сертификат пен жеке кілттің міндеті бөлек: сертификат таратылуы мүмкін, ал жеке кілт иесінің бақылауында қалады.

Зертханалық сертификатты дайындау

Төмендегі команданы OpenSSL орнатылған терминалда бір жол ретінде орындаңыз. cert.pem және key.pem файлдары ағымдағы каталогта жасалады. -nodes жеке кілтті зертхана үшін құпиясөзсіз жазады; файлды жарияламаңыз және ортақ бумаға салмаңыз. Екі күндік мерзім осы тәжірибеге арналған, сертификат беру саясатын сипаттамайды.

OpenSSL командасы

```
openssl req -x509 -newkey rsa:2048 -nodes -days 2 -keyout key.pem -out cert.pem -subj "/CN=localhost" -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"
```

Команда Windows жүйесінде OpenSSL бар терминалда да орындалады. Сертификатты жасамай серверді іске қоссаңыз, файл табылмау қатесі шығады. Қате атауға берілген сертификатпен тексеру тәжірибесінде клиенттің server_hostname параметрін басқа атауға өзгертіңіз; қосылым сертификат тексеруінен өтпеуі тиіс.

TLS echo сервері

week11_server.py

```
import socket
import ssl
from protocol import recv_message, send_message

context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.minimum_version = ssl.TLSVersion.TLSv1_2
context.load_cert_chain("cert.pem", "key.pem")

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
    server.bind(("127.0.0.1", 9443))
    server.listen(5)
    print("TLS server on 127.0.0.1:9443")
    while True:
        raw, address = server.accept()
        with raw:
            raw.settimeout(5)
            try:
                with context.wrap_socket(raw, server_side=True) as conn:
                    request = recv_message(conn)
                    send_message(conn, {"ok": True, "echo": request})
            except (OSError, ValueError, EOFError, RecursionError) as error:
                print(type(error).__name__)
```

week11_client.py

```
import socket
import ssl
from protocol import recv_message, send_message

context = ssl.create_default_context(cafile="cert.pem")
with socket.create_connection(("127.0.0.1", 9443), timeout=5) as raw:
    with context.wrap_socket(raw, server_hostname="localhost") as conn:
        send_message(conn, {"text": "Қорғалған хабар"})
        print("TLS:", conn.version())
        print(recv_message(conn))
```

protocol.py, cert.pem және key.pem сервер каталогында орналасады. Клиентке protocol.py мен cert.pem жеткілікті; жеке кілт клиентке көшірілмейді. Іске қосу: python week11_server.py және python week11_client.py. Күтілетін нәтиже — келісілген TLS нұсқасы және ok мәні True жауап. Нақты келісілген нұсқа орнатылған Python мен OpenSSL қолдауына тәуелді.

Қолданба деңгейіндегі қорғаныс

Аутентификация токенін URL ішіне немесе журналға жазбаңыз. Токеннің қолданылу мерзімі, аудиториясы және құқықтары тексеріледі. Құпия сөз сақтауда қарапайым SHA-256 қолдану жеткіліксіз: құпия сөзге арналған тұзды, есептеу құнын басқаратын арнайы алгоритм қажет. Ал файлдың тұтастығын тексеруде SHA-256 орынды; міндеттері әртүрлі.

Желі арқылы келген pickle дерегін ашу қауіпті: десериализация код орындауға әкелуі мүмкін. JSON сияқты дерек пішімі таңдалса да, көлем, тереңдік және өріс мағынасы тексеріледі. Файл атауы ретінде келген ../ сияқты жолдарды тікелей сақтау жолына қосуға болмайды. Пайдаланушы берген атаудың орнына сервер жасаған идентификаторды қолдану жолды басқаруды жеңілдетеді.

Қарқын шегі, дене көлемі, белсенді қосылым саны және жалпы мерзім қолжетімділікті қорғауға қатысады. Бұл шаралар аутентификациямен бірге жұмыс істейді. Сервер қате жауабында ішкі трассаны бермейді, ал журналда оқиғаны құпиясыз идентификатормен байланыстырады.

Бақылау сұрақтары

1. TLS пен авторизация неге бөлек?
2. server_hostname қандай міндет атқарады?
3. CERT_NONE қандай қорғанысты алып тастайды?
4. Сертификат пен жеке кілт қай тараптарға беріледі?

Өздік жұмыс: дұрыс сертификат, қате хост атауы және сенілмейтін сертификат жағдайларын салыстырыңыз. Әрқайсысына алынған ерекшелік атауын жазыңыз. Қосымша тапсырма: файл жүктеу қызметі үшін құпияны қорғау, жолды тексеру және көлемді шектеу талаптарын жасаңыз.

Дәрісті өткізу реті: қауіпсіздік міндеттеріне 20 минут, сертификатқа 20 минут, TLS мысалына 30 минут, қолданба қорғанысына 10 минут, бекітуге 10 минут.

12 Дәріс. Үзілістер уақыт шектеуі және қайталау саясаты

Дәріс мақсаты: желілік ақау кезінде сұраудың тағдырын бағалап, қайталауды шектеу мен күйдің дұрыстығын сақтауды үйрету.

Оқу нәтижелері: студент операция мерзімін есептейді, уақытша және тұрақты қатені ажыратады, backoff пен jitter қолданады және идемпотентті сұрауды жобалайды.

Ақаудың белгісіздігі

Жергілікті функция ерекшелік көтергенде оның аяқталу шекарасы көрінеді. Желілік сұрауда жауап жоғалса, сервер әрекетті орындап үлгерген-үлгермегені белгісіз қалуы мүмкін. Клиент тапсырыс құру сұрауын жіберді, сервер тапсырысты сақтады, бірақ жауап жолда жоғалды. Клиент қайта жіберсе, екінші тапсырыс жасалуы мүмкін. Уақыт шектеуінің аяқталуы «сервер ештеңе істемеді» деген дәлел емес.

Осы себеппен ақауды үш сұрақ арқылы талдаймыз: сұрау жіберілді ме, сервер орындады ма, нәтиже клиентке жетті ме? Бірінші сұрақтың жауабы екіншісін анықтамайды. Журнал мен сұрау идентификаторы екі тараптың оқиғаларын байланыстыруға көмектеседі. Түпкілікті күйді тексеретін жеке оқу сұрауы белгісіздікті шешудің бір жолы.

Әрекет мерзімі және жалпы уақыт бюджеті

Қосылу, оқу және жазу мерзімдері әртүрлі кезеңді шектейді. Сокетке 5 секунд timeout орнату бірнеше recv шақыруынан тұратын хабардың жалпы уақыты 5 секундтан аспайтынын білдірмейді. Қарсы жақ әр 4 секунд сайын бір байт берсе, әр оқу үлгеріп, бүкіл әрекет ұзаққа созылады. Қатаң шек қажет болса, бір абсолютті deadline есептеліп, әр келесі кезеңге қалған уақыт беріледі.

Уақыт бюджеті атау шешу, қосылу, TLS, кезекте күту, жауап оқу және қайталау үзілістерін қамтиды. Ұзақ міндетті қызметтер тізбегінде әр ішкі шақыруға бастапқы толық timeout беру жалпы кідірісті өсіреді. Бастапқы сұраудың қалған мерзімі төменгі қызметтерге таратылады. Өткен уақытты time.monotonic арқылы өлшеу жүйе сағаты өзгерісінің әсерін жояды.

Қайталау қай кезде орынды

Қате пішім, рұқсаттың жоқтығы немесе қолдамайтын операция сұрауды өзгертпей қайталанғанда түзелмейді. Уақытша қосылым үзілуі, бос емес қызмет немесе келісілген уақытша мәртебе шектеулі қайталауға негіз бола алады. Әр HTTP 5xx коды автоматты қайталауға бірдей қолайлы емес; қызмет келісімі және әдістің мағынасы ескеріледі. Сервер берген Retry-After нұсқауы болса, клиент оны өз уақыт бюджетімен үйлестіріп қолданады.

Backoff әрекеттер арасындағы күтуді ұлғайтады. Jitter сол аралыққа кездейсоқтық енгізіп, көптеген клиенттің бір сәтте қайта жиналуын азайтады. Кездейсоқтық әрекет саны мен жалпы мерзім шегін алмастырмайды. Бірнеше қабатта үш реттен қайталау $3 \times 3 \times 3 = 27$ төменгі сұрауға дейін көбейе алады; қай қабат қайталауды басқаратыны айқын жазылады.

Оқуға арналған шектелген қайталау мысалы

Код нақты желілік кітапханаға байланбайды. operation қалған секундты қабылдап, оны өз желілік әрекеттеріне қолдануы тиіс. Мысалда алғашқы екі әрекет TimeoutError береді, үшінші әрекет сәтті аяқталады. Автоматты қайталау тек қайталауға қауіпсіз оқу әрекетіне арналған.

week12_retry.py

```
import random
import time

def retry_read(operation, attempts=3, budget=4.0):
    if attempts < 1 or budget <= 0:
        raise ValueError("Positive limits required")
    deadline = time.monotonic() + budget
    for index in range(attempts):
        remaining = deadline - time.monotonic()
        if remaining <= 0:
            raise TimeoutError("Total budget exhausted")
        try:
            result = operation(remaining)
            if time.monotonic() > deadline:
                raise TimeoutError("Total budget exhausted")
            return result
        except (TimeoutError, ConnectionError):
            if index == attempts - 1:
                raise
            remaining = deadline - time.monotonic()
            if remaining <= 0:
                raise TimeoutError("Total budget exhausted")
            pause = min(0.2 * (2 ** index), remaining)
            time.sleep(random.uniform(0, pause))

if __name__ == "__main__":
    state = {"calls": 0}
    def demo(remaining):
        state["calls"] += 1
        print("attempt", state["calls"])
        if state["calls"] < 3:
            raise TimeoutError("Temporary delay")
        return {"status": "ok"}
    print(retry_read(demo))
```

Іске қосу: python week12_retry.py. Нәтиже: attempt 1, attempt 2, attempt 3 және status мәні ok нысаны. Бұл қабықша синхронды operation функциясын күшпен тоқтата алмайды. Сондықтан operation өз күтуін remaining шегінде орындауы міндетті; сыртқы қабықша мерзімнен асқан нәтижені ғана қабылдамай алады. Осы шектеуді жасыру клиенттің уақыт кепілдігін бұрмалайды.

Идемпотенттілік кілті және транзакция

Күй өзгертетін сұрауға клиент тұрақты операция кілтін жасайды. Бір логикалық әрекетті қайталағанда сол кілт сақталады. Сервер кілтті сұрау мазмұнының ізімен және нәтижемен байланыстырады. Бір кілт басқа мазмұнмен келсе, сервер қақтығысты қайтарады. Бірдей кілтке нәтиже сақтау мен негізгі әрекет атомарлы орындалмаса, екеуінің аралығындағы ақау қайталануды қайта тудырады.

Кілттерді шексіз сақтау жадты өсіреді, сондықтан сақтау мерзімі мен сыйымдылығы анықталады. Кілт ұмытылғаннан кейінгі ескі қайталаудың мінезі келісімде көрсетіледі. Бір процесс жадындағы кэш қайта іске қосылудан кейін сақталмайды. Тұрақты кепіл үшін дерекқордағы бірегей шектеу мен транзакция қажет.

Ақауды оқшаулау

Circuit breaker істен шыққан тәуелділікке үзіліссіз сұрау жіберуді уақытша тоқтатады. Жабық күйде сұраулар өтеді, ашық күйде жылдам қайтарылады, жартылай ашық күйде аз сынақ сұрауы жіберіледі. Бұл тетік қателікті

жоймайды; жүйенің қалған бөлігін ұзақ күтуден қорғауға көмектеседі. Оның шегі, қалпына келу аралығы және көрсеткіштері өлшеуге сүйенуі тиіс.

Бақылау сұрақтары

1. Timeout сервердің әрекет жасамағанын дәлелдей ме?
2. Жеке recv timeout пен жалпы deadline қалай ажыратылады?
3. Jitter қандай топтық әсерді азайтады?
4. Идемпотенттілік кілтін сақтауды негізгі әрекетпен неге біріктіру керек?

Өздік жұмыс: «тапсырманы тапсыру» операциясына idempotency_key қосылған келісім жазыңыз. Жауап жоғалған, бір кілт екі рет келген және бір кілт әртүрлі мазмұнмен келген жағдайларды кестеге түсіріңіз. Қосымша есеп: үш әрекеттің әрқайсысы 1 секунд, аралықтары 0,2 және 0,4 секунд болса, жалпы уақыт 3,6 секундқа тең; 3 секунд бюджетке сыйғызу үшін соңғы әрекеттің шегін есептеңіз.

Дәрісті өткізу реті: ақау белгісіздігіне 20 минут, мерзімге 20 минут, қайталау кодына 25 минут, идемпотенттілікке 15 минут, бекітуге 10 минут.

13 Дәріс. Файлдарды тасымалдау және деректердің тұтастығы

Дәріс мақсаты: үлкен деректі бөліктермен жіберіп, көлемін, мазмұнын және сақтау жолын тексеретін файл қабылдау қызметін құру.

Оқу нәтижелері: студент метадерек пен файл байттарын ажыратады, ағындық көшіру орындайды, SHA-256 ізін салыстырады және уақытша файлмен жұмыс істейді.

Файл тасымалының келісімі

Файлды бір sendall арқылы жіберу шағын мысалда ыңғайлы көрінгенімен, бүкіл файлды жадқа оқу үлкен көлемде тиімсіз. Ағындық көшіру тұрақты көлемдегі бөліктерді кезекпен оқып, жібереді. Буфердің 64 КиБ болуы файл көлемі 1 МиБ не 10 МиБ болса да бір қадамдағы жад шығынын шектейді. Жалпы жадқа қосылым саны мен қосымша буферлер де әсер етеді.

Қабылдаушы файлдың аяқталуын TCP қосылымының кездейсоқ үзілуінен ажыратуы керек. Осы дәрісте алдымен ұзындығы префикспен берілген JSON метадерек жіберіледі. Онда файл көлемі мен SHA-256 ізі бар. Сервер метадеректі тексеріп, ready жауабын береді. Клиент содан кейін дәл көрсетілген көлемдегі файл байттарын жібереді. Сервер көлем мен ізді тексергеннен кейін сақтау идентификаторын қайтарады.

Кезең	Жіберуші	Мазмұны	Қабылдаушы тексеруі
Метадерек	Клиент	size және sha256	Көлем шегі және із пішімі
Рұқсат	Сервер	ready мәні	Файл жіберуге келісім
Файл	Клиент	Дәл size байт	Ерте үзілу және мерзім
Нәтиже	Сервер	ok және file_id	Сақтау нәтижесі

Ready жауабы клиентті қабылданбайтын файлды босқа жіберуден сақтайды. Дегенмен клиент қате әрекет етуі мүмкін, сондықтан сервер әр кезеңді өзі шектейді. Бір қосылымда осы операция бір рет орындалады, соңынан қосылым жабылады. Артық жіберілген байттар келесі операция ретінде өңделмейді.

Бақылау ізі және сенім

SHA-256 дерек мазмұнынан 256 биттік із есептейді. Бір файлды бөліктермен update арқылы өңдеу оны біртұтас берумен бірдей із береді. Python hashlib осы ағындық интерфейсті ұсынады. Бұл тәсіл файлды жадқа толық жинамай тұтастықты тексеруге мүмкіндік береді. [23]

Клиент хабарлаған із бен сервер есептеген іздің теңдігі тасымалданған мазмұнның клиент мәлімдеген мазмұнға сәйкес екенін тексереді. Ол файл авторының кім екенін дәлелдемейді. Қарсы тарап файл мен ізді бірге ауыстыра алса, хэш салыстыруы оны анықтамайды. Арнаның түпнұсқалығы мен құпиялығы TLS арқылы, ал мазмұнның дербес авторлығы қажет болса цифрлық қолтаңба арқылы қорғалады.

Уақытша файл және қауіпсіз атау

Клиент берген файл атауын сервер жолына тікелей қосу жолдан шығу мәселесін тудырады. Бұл мысалда сервер өз UUID идентификаторын жасап, қабылданған файлды сол атаумен сақтайды. Қабылдау алдымен сол каталогтағы уақытша файлға жүреді. Көлем мен хэш дұрыс болса, os.replace уақытша файлды соңғы атауға ауыстырады. Үзілген файл соңғы ресурс ретінде көрінбейді.

Атын ауыстырудың атомарлығы мен электр қуаты үзілгеннен кейінгі тұрақтылық бір ұғым емес. Операциялық жүйеге жазылған дерек дискіге түпкілікті бекімеуі мүмкін. Тұрақты сақтау кепілдігі қажет қызмет fsync және файлдық жүйенің тәртібін ескереді. Бұл мысал қалыпты процесс жұмысы кезіндегі бүтін файлды жариялауды көрсетеді.

Файл қабылдайтын сервер

week13_server.py

```
import hashlib
import os
from pathlib import Path
import socket
import tempfile
import time
import uuid
from protocol import recv_message, send_message

ROOT = Path("uploads")
ROOT.mkdir(exist_ok=True)
MAX_FILE = 10 * 1024 * 1024

def receive_file(conn):
    conn.settimeout(5)
    meta = recv_message(conn)
    size, expected = meta.get("size"), meta.get("sha256")
    if type(size) is not int or not 0 <= size <= MAX_FILE:
        raise ValueError("Invalid size")
    if not isinstance(expected, str) or len(expected) != 64:
        raise ValueError("Invalid digest")
    if any(c not in "0123456789abcdef" for c in expected):
        raise ValueError("Invalid digest")
    temp_path = None
    try:
        with tempfile.NamedTemporaryFile(dir=ROOT, delete=False) as target:
            temp_path = Path(target.name)
            send_message(conn, {"ready": True})
            remaining = size
            digest = hashlib.sha256()
            deadline = time.monotonic() + 30
            while remaining:
                budget = deadline - time.monotonic()
                if budget <= 0:
                    raise TimeoutError("Upload deadline")
                conn.settimeout(min(5, budget))
                block = conn.recv(min(65536, remaining))
                if not block:
                    raise EOFError("Incomplete file")
                target.write(block)
                digest.update(block)
                remaining -= len(block)
        if digest.hexdigest() != expected:
            raise ValueError("Digest mismatch")
        file_id = uuid.uuid4().hex
        os.replace(temp_path, ROOT / file_id)
        conn.settimeout(5)
        send_message(conn, {"ok": True, "file_id": file_id})
    finally:
        if temp_path is not None:
            temp_path.unlink(missing_ok=True)

if __name__ == "__main__":
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server:
        server.bind(("127.0.0.1", 9004))
        server.listen(5)
```

```
    print("Upload server on 127.0.0.1:9004")
    while True:
        conn, address = server.accept()
        with conn:
            try:
                receive_file(conn)
            except (OSError, ValueError, EOFError, RecursionError) as error:
                print(type(error).__name__)
```

Файл жіберетін клиент

week13_client.py

```
import hashlib
import os
from pathlib import Path
import socket
import sys
from protocol import recv_message, send_message

source = Path(sys.argv[1] if len(sys.argv) > 1 else "sample.txt")
with source.open("rb") as stream:
    size = os.fstat(stream.fileno()).st_size
    if not 0 <= size <= 10 * 1024 * 1024:
        raise ValueError("File exceeds course limit")
    digest = hashlib.sha256()
    while block := stream.read(65536):
        digest.update(block)
    stream.seek(0)
    with socket.create_connection(("127.0.0.1", 9004), timeout=5) as conn:
        send_message(conn, {"size": size, "sha256": digest.hexdigest()})
        if recv_message(conn).get("ready") is not True:
            raise ValueError("Server rejected upload")
        remaining = size
        while remaining:
            block = stream.read(min(65536, remaining))
            if not block:
                raise EOFError("Source file changed")
            conn.sendall(block)
            remaining -= len(block)
        print(recv_message(conn))
```

protocol.py осы файлдармен бір каталогта орналасады. sample.txt атты қысқа мәтіндік файл жасаңыз. Іске қосу: python week13_server.py, кейін python week13_client.py sample.txt. Нәтижеде file_id беріледі, uploads каталогында сол атаулы файл пайда болады. Клиент екі рет оқиды: алдымен із есептейді, кейін жібереді. Файл осы аралықта өзгерсе, сервердегі із сәйкес келмейді және уақытша файл жойылады.

Шектеулер және дамыту жолдары

Сервер бір қосылымды бір уақытта өңдейді және әр файлға 10 МиБ шек қояды. Бірақ сақталған файлдардың жалпы көлеміне квота енгізілмеген. Көп шағын файл да дискіні толтыра алады; нақты қызметте пайдаланушы квотасы, жалпы бос орын және сақтау мерзімі бақыланады. Кіріс метадерегінің жеке оқу timeout мәні бүкіл метадеректің жалпы мерзімін шектемейді, сондықтан сыртқы қызметке айналдырғанда толық deadline қосылады.

Қайта жалғастырылатын тасымалдау файл идентификаторын, қабылданған ығысуды және бөліктің тұтастығын тексеруді талап етеді. Клиент «осы байттан жалғастыр» деген мәлімдемеге соқыр сенбейді; сервер бұрынғы

мазмұнның сәйкестігін тексереді. Жауап жоғалса, аяқталған файлдың мәртебесін id арқылы қайта сұрау операцияны екі рет орындауды азайтады.

Бақылау сұрақтары

1. Неге бүкіл файлды жадқа оқу қажет емес?
2. Хэш авторлықты растай ма?
3. Уақытша файл қандай мәселені шешеді?
4. Бір файл көлемінің шегі неге жалпы квотаны алмастырмайды?

Өздік жұмыс: бос файл, 1 МиБ файл және қате хэші бар файлды тексеріңіз. Дұрыс қабылданған файлдардың іздерін салыстырыңыз. Клиентті тасымал ортасында тоқтатып, соңғы атауда жартылай файл қалмайтынын тексеріңіз. Қосымша жобалау тапсырмасы: мәртебені қайта сұрайтын операция мен жалпы 100 МиБ сақтау квотасын сипаттаңыз.

Дәрісті өткізу реті: келісімге 20 минут, тұтастық пен сақтауға 20 минут, сервер мен клиентке 30 минут, шектеулерге 10 минут, бекітуге 10 минут.

14 Дәріс. Желілік бағдарламаларды тестілеу және диагностика

Дәріс мақсаты: қалыпты және ақаулы жағдайларды жоспарлы тексеріп, журнал мен өлшем арқылы мәселенің себебін анықтау.

Оқу нәтижелері: студент модульдік және интеграциялық сынақты ажыратады, бөлшектенген хабарға тест жазады, кідіріс көрсеткіштерін есептейді және диагностика ретін қолданады.

Тексеру деңгейлері

Модульдік тест хаттама парсері сияқты шағын бөлікті сыртқы серверсіз тексереді. Интеграциялық тест нақты клиент пен сервердің бірге жұмысын тексереді. Жүктеме сынағы белгілі қарқындағы сұраулар кезінде кідіріс, қате және ресурс өзгерісін өлшейді. Төзімділік сынағы жүйенің ұзақ жұмысында жад, дескриптор немесе кезек жиналуын іздейді. Бұл тексерулер бірін-бірі алмастырмайды.

Желілік кодтың маңызды қасиеті — бөлшектенуге тәуелсіздік. Бір хабар бірден немесе ұсақ бөліктермен келсе, нәтиже бірдей болуы тиіс. Оны нақты желідегі кездейсоқ сегменттелуге сеніп тексеру жеткіліксіз. Тест сокет орнына бақыланатын fake нысанын беріп, recv қайтарған бөліктер көлемін анық басқара алады. Мұндай тест желіні емес, парсер келісімін тексереді.

Хаттамаға арналған модульдік тесттер

Төмендегі файл 5 дәрістегі protocol.py модулімен бір каталогта сақталады. FakeSocket әр оқуда белгіленген шағын көлемнен артық байт бермейді. Сынақтар UTF-8 мәтінін, біріктірілген кадрларды, қысқа денені және артық ұзындықты тексереді. unittest ішіндегі assertEqual нәтиженің күтілген мәнге сәйкестігін, assertRaises қажетті ерекшеліктің туындауын тексереді. [13]

test_protocol.py

```
import struct
import unittest
from protocol import MAX_BODY, encode_message, recv_message

class FakeSocket:
    def __init__(self, data, step=1):
        self.data = bytearray(data)
        self.step = step

    def recv(self, count):
        take = min(count, self.step, len(self.data))
        result = bytes(self.data[:take])
        del self.data[:take]
        return result

class ProtocolTests(unittest.TestCase):
    def test_fragmented_utf8(self):
        message = {"text": "Ә Ғ Қ Ң Ө Ұ Ү Һ І"}
        sock = FakeSocket(encode_message(message), step=1)
        self.assertEqual(recv_message(sock), message)

    def test_two_frames(self):
        first, second = {"id": 1}, {"id": 2}
        wire = encode_message(first) + encode_message(second)
        sock = FakeSocket(wire, step=1000)
        self.assertEqual(recv_message(sock), first)
        self.assertEqual(recv_message(sock), second)
```

```
    def test_incomplete_body(self):
        wire = encode_message({"id": 1})[:-1]
        with self.assertRaises(EOFError):
            recv_message(FakeSocket(wire, step=3))

    def test_oversized_header(self):
        wire = struct.pack("!I", MAX_BODY + 1)
        with self.assertRaises(ValueError):
            recv_message(FakeSocket(wire))

    def test_invalid_utf8(self):
        wire = struct.pack("!I", 1) + b"\xff"
        with self.assertRaises(UnicodeDecodeError):
            recv_message(FakeSocket(wire))

if __name__ == "__main__":
    unittest.main()
```

Іске қосу: python -m unittest -v test_protocol.py. Бес тесттің бәрі OK нәтижесімен аяқталуы тиіс. FakeSocket ішінде send жоқ, себебі осы тест тек оқу жолын тексереді. Жіберу логикасын тексергенде бөлек нысан байттарды жинап, алынған кадрды күтілген мәнмен салыстырады. Тесттің мақсаты нақты талапты тексеру; іске асыру жолдарын қайталап көшіру емес.

Интеграциялық сценарийлер

Сценарий	Күтілетін нәтиже	Тексерілетін қасиет
Дұрыс қысқа сұрау	Келісілген жауап	Негізгі функция
Бір байттан келетін кадр	Дұрыс бүтін хабар	Бөлшектенуге төзімділік
Қате ұзындық	Бас тарту және ресурсты жабу	Жад шегі
Клиенттің ерте жабылуы	Бақыланатын EOF не қате	Ресурс тазалау
Үнсіз клиент	Мерзімнен кейін аяқтау	Күту шегі
Бірнеше қатар клиент	Жауаптардың араласпауы	Оқшаулау
Қате TLS атауы	Сертификат қатесі	Түпнұсқалықты тексеру
Қате файл хэші	Соңғы файлды жарияламау	Тұтастық

Автоматты интеграциялық тестте тұрақты бос портқа сенудің орнына bind кезінде 0 беріп, жүйе таңдаған портты getsockname арқылы клиентке жеткізу ыңғайлы. Сервердің іске қосылғанын тек sleep арқылы күту тұрақсыз: баяу жүйеде уақыт жетпей қалады. Дайындық белгісі немесе шектелген қосылу тексеруі қолданылады. Әр тест соңында процесс, сокет және уақытша файлдар тазаланады.

Журналдау және бақылау көрсеткіштері

Журнал оқиғаның егжей-тегжейін береді: уақыт, деңгей, сұрау идентификаторы, операция, мәртебе, ұзақтық және қате санаты. Метрика көптеген оқиғаны сандық түрде жинақтайды: сұрау саны, қате үлесі, белсенді қосылым, кезек ұзындығы, қабылданған байт. Таралған трасса бір сұраудың бірнеше қызмет арқылы өтуін байланыстырады. Осы үш құрал әртүрлі сұраққа жауап береді.

Python logging модулі деңгейлерді, пішімдеуді және әртүрлі шығыс өңдегіштерін қолдайды. logging.exception except блогында трассаны тіркей алады. Токен, құпия сөз және толық жеке деректерді журналға енгізбеу қажет. Журналдағы мәлімет жұмысқа жеткілікті, бірақ артық құпияны ашпайтын деңгейде таңдалады. [21]

Кідірісті өлшеу мысалы

Келесі клиент 9 дәрістегі /health жолына 20 тізбекті сұрау жібереді. Бұл жүктеме генераторы емес; өлшеуді дұрыс тіркеуге арналған шағын мысал. Қате сұрау үлгіден үнсіз алынып тасталмайды, бөлек саналады. Үлгі көлемі аз болғандықтан p95 тұрақты қызмет көрсеткішін дәл бағалауға жеткіліксіз.

week14_measure.py

```
import math
import statistics
import time
from urllib.request import urlopen

durations = []
errors = 0
for _ in range(20):
    start = time.perf_counter()
    try:
        with urlopen("http://127.0.0.1:8001/health", timeout=2) as response:
            response.read(4096)
        durations.append((time.perf_counter() - start) * 1000)
    except OSError:
        errors += 1

print("success:", len(durations), "errors:", errors)
if durations:
    ordered = sorted(durations)
    p95 = ordered[math.ceil(0.95 * len(ordered)) - 1]
    print("median_ms:", round(statistics.median(ordered), 3))
    print("p95_ms:", round(p95, 3))
```

Іске қосу: 9 дәрістің серверін іске қосып, python week14_measure.py орындаңыз. Уақыт мәндері компьютерге байланысты өзгереді. Медиана орталық кідірісті көрсетеді, p95 үлгілердің 95 пайызы орналасатын шекараны жуықтайды. Орташа мән бірнеше ұзақ сұраудың немесе құйрық кідірістің әсерін жеке өзі ашпайды. Өлшеу есебінде клиент саны, хабар көлемі, тест ұзақтығы және орта сипаттамасы беріледі.

Диагностика реті

Ақауды қолданбадан төменге немесе желіден жоғарыға бірізді тексеріңіз. Алдымен күтілетін хост пен портты тіркеңіз. DNS нәтижесін қараңыз. Сервердің тыңдап тұрғанын операциялық жүйе құралымен тексеріңіз. Одан кейін TCP не UDP алмасуын, TLS келісімін және қолданбалық мәртебені тексеріңіз. ping жауабы порттың ашықтығын дәлелдемейді; ping жауабының жоқтығы да HTTP қызметі міндетті түрде істемейді деген қорытынды бермейді.

Пакет түсірілімі бастапқы және мақсат мекенжайды, портты, уақытты және қайта жеткізуді зерттеуге көмектеседі. Тек өз зертханаңыздағы трафикті жинаңыз. TLS қолданылса, қолданба мәтінінің ашық көрінбеуі қалыпты. Шифрды өшірудің орнына өз қолданбаңыздағы қауіпсіз журналдау мен өлшемді пайдаланыңыз.

Бақылау сұрақтары

1. FakeSocket қандай тәуелділікті алып тастайды?
2. Бір нақты желілік тест бөлшектенуді неге дәл тексермейді?
3. Медиана мен p95 қандай айырманы көрсетеді?
4. ping неге қолданба қолжетімділігінің толық сынағы емес?

Өздік жұмыс: test_protocol.py ішіне нөл ұзындық және JSON массиві үшін екі теріс тест қосыңыз. Файл серверіне арналған сегіз сценарийлік сынақ жоспарын жазыңыз. 20 сұрау өлшемін үш рет қайталап, нәтиже өзгерісін қорытындылаңыз; өлшенбеген өткізу қабілеті туралы тұжырым жасамаңыз.

Дәрісті өткізу реті: тест деңгейлеріне 15 минут, модульдік сынаққа 25 минут, интеграцияға 20 минут, өлшем мен диагностикаға 20 минут, бекітуге 10 минут.

15 Дәріс. Желілік қызметті біріктіру және қорытынды жоба

Дәріс мақсаты: курс компоненттерін бір қызметке біріктіріп, талап, іске қосу, бақылау және қорғаныс шешімдерін негіздеу.

Оқу нәтижелері: студент желілік жобаның архитектурасын сипаттайды, конфигурация мен құпияны ажыратады, реттелген тоқтатуды жоспарлайды және тексерілетін қабылдау өлшемдерін ұсынады.

Талаптан архитектураға өту

Жобалау пайдаланушы әрекетінен басталады. «Файл сервері керек» деген талап жеткіліксіз. Кім файл жібереді, көлемі қандай, бір уақытта қанша клиент келеді, жауап қанша уақытта қажет, дерек қанша уақыт сақталады және файлға кім қатынайды деген сұрақтар шешіледі. Жауаптар хаттама, сақтау, қатар орындау және қауіпсіздік тетіктерін таңдауға негіз береді.

Функциялық талап жүйенің не істейтінін сипаттайды: файл қабылдау, мәртебе қайтару, жазбаны оқу. Сапа талабы әрекеттің шегін көрсетеді: файл 10 МиБ-тан аспайды, қате сұрау процесс жұмысын тоқтатпайды, 20 қатар клиентте жауаптар араласпайды. «Жылдам» немесе «сенімді» деген жалғыз сөздер сынаққа жеткіліксіз. Әр талапқа өлшем мен тексеру әдісі беріледі.

Қабаттарға бөлу кодтың өзгерісін оқшаулайды. Тасымалдау қабаты сокет пен TLS-ті басқарады. Хаттама қабаты кадрлау мен валидацияны орындайды. Қызмет қабаты операцияны жүзеге асырады. Сақтау қабаты файл не дерекқор күйіне жауап береді. Бақылау қабаты журнал мен метриканы жинайды. Бір файлға бәрін біріктіру бастапқы мысалда түсінікті, бірақ жоба ұлғайғанда бөлек функциялар мен модульдерге бөлу қажет.

Конфигурация және іске қосу

Хост, порт, көлем шегі және дерек каталогы кодтағы кездейсоқ тұрақтыларға айналмауы тиіс. Конфигурация іске қосу кезінде оқылып, бір рет тексеріледі. Қате порт немесе теріс көлем анықталса, сервер сұрау қабылдамай тұрып тоқтайды. Құпия токен мен жеке кілт репозиторийге қосылмайды. Конфигурацияны журналға түгел шығару құпияны жариялауы мүмкін, сондықтан тек құпия емес мәндер тіркеледі.

week15_config.py

```
from dataclasses import dataclass
import os
from pathlib import Path

@dataclass(frozen=True)
class Settings:
    host: str
    port: int
    max_file: int
    data_dir: Path

def load_settings():
    port = int(os.getenv("NETCOURSE_PORT", "9004"))
    max_file = int(os.getenv("NETCOURSE_MAX_FILE", "10485760"))
    if not 1024 <= port <= 65535:
        raise ValueError("Port must be in 1024..65535")
    if not 1 <= max_file <= 100 * 1024 * 1024:
        raise ValueError("Invalid file limit")
    return Settings(
        host=os.getenv("NETCOURSE_HOST", "127.0.0.1"),
        port=port,
        max_file=max_file,
```

```
        data_dir=Path(os.getenv("NETCOURSE_DATA_DIR", "uploads")),
    )

if __name__ == "__main__":
    settings = load_settings()
    print("listen:", settings.host, settings.port)
    print("max_file:", settings.max_file)
    print("data_dir:", settings.data_dir)
```

Іске қосу: python week15_config.py. Әдепкі нәтиже 127.0.0.1, 9004, 10485760 және uploads мәндерін береді. Модульді 13 дәрістің серверіне импорттап, ROOT, MAX_FILE және bind параметрлерін settings өрістерімен байланыстырыңыз. frozen=True конфигурацияның кездейсоқ өзгеруін шектейді; бұл құпияны қорғау тетігі емес.

Реттелген тоқтату және дайындық

Процесті күрт тоқтату жазылып жатқан файлды, жауап кезегін немесе журналды аяқталмай қалдыруы мүмкін. Реттелген тоқтату төрт әрекетті біріктіреді: жаңа жұмысты қабылдауды тоқтату, ағымдағы жұмысқа шектелген аяқтау уақытын беру, қалған міндеттерді тоқтату, ресурстарды жабу. Аяқтау мерзімі шексіз емес; үнсіз клиент қызметті мәңгі ұстап қалмауы тиіс.

Liveness процесстің тірі екенін тексереді, readiness жаңа сұрау қабылдауға дайындығын тексереді. Дерекқорға қатынау жоқ кезде процесс тірі қалуы мүмкін, бірақ жазу операциясына дайын емес. Екеуін бір тексеруге біріктіру қажетсіз қайта іске қосу тізбегін тудыруы мүмкін. Дайындық жауабы тәуелділіктің нақты талапқа әсеріне сүйенеді.

Кері прокси TLS аяқтауын, маршруттауды және кейбір шектерді басқара алады. Прокси бар болса да, қолданба дене көлемі мен рұқсатты тексеруді сақтайды. Прокси жіберген клиент мекенжайы тақырыбына тек сенімді проксиден келгенде сенуге болады; сыртқы клиент оны өзі жазып жібере алады. Тікелей ішкі портқа қатынау саясаты орналастыру сызбасымен бірге анықталады.

Өнімділік және хаттама таңдауы

Көлденең масштабтау бірнеше процесс не түйін қосады. Егер сеанс күйі тек бір процесс жадында тұрса, келесі сұрауды басқа процесс өңдегенде күй табылмауы мүмкін. Ортақ сақтау, дербес сұрау келісімі немесе саналы маршруттау шешімі қажет. Жүктемені теңестіру баяу дерекқорды өздігінен жылдамдатпайды; шектеуші буын өлшеніп анықталады.

HTTP/2 бір TCP қосылымында бірнеше ағынды көбейтіп пайдалануға мүмкіндік береді, бірақ TCP деңгейіндегі жоғалған байт кейінгі байттардың жеткізілуін тежей алады. QUIC тәуелсіз ағындармен жұмыс істеп, бір ағын ішіндегі реттілікті қамтамасыз етеді; пакет жоғалуы басқа ағынның дерек ретін дәл сол тәсілмен тоқтатпайды. Дегенмен ортақ кептеліс бақылауы мен арна шектеуі барлық тасымалға әсер етеді. Хаттаманы ауыстыру қолданбадағы шексіз кезек пен қате келісімді жоймайды. [16]

Қорытынды жоба тапсырмасы

Негізгі жоба — оқу файлдарын қабылдау қызметі. 13 дәрістегі тасымалдау логикасын, 11 дәрістегі TLS қорғанысын, 5 дәрістегі хабар пішімін және 14 дәрістегі сынақтарды біріктіріңіз. Бір компьютердегі толық көрсетілім жеткілікті. Қызметті интернетке жариялау талап етілмейді. Сервер мен клиентті басқа студент нұсқаулық бойынша іске қоса алуы тиіс.

Жобаның міндетті әрекеттері: файл метадерегін қабылдау; көлем мен хэшті тексеру; уақытша файлды аяқталған ресурсқа ауыстыру; сұрау идентификаторын қайтару; қате жағдайын машиналық кодпен көрсету;

TLS атауын тексеру; бос тұрған қосылымға мерзім қою; файл мен жалпы сақтау көлемін шектеу. Бірнеше клиентке қызмет көрсету ағындар пулы немесе asyncio арқылы іске асырылады.

Қайталанған жүктеу сұрауына арналған саясатты таңдаңыз. Бірдей операция кілті мен бірдей файл ізі келсе, бұрынғы нәтижені қайтару ұсынылады. Бірдей кілт басқа ізбен келсе, қақтығыс жауабы беріледі. Жобада тек жадтағы кэш қолданылса, қайта іске қосылғаннан кейінгі шектеуі есепте ашық көрсетіледі. Тұрақты қайталау кепілдігі қосымша дерекқор транзакциясын талап етеді.

Қабылдау талабы	Көрсету тәсілі
Дұрыс файл өзгеріссіз сақталады	Клиент пен сервер хэшін салыстыру
Шектен үлкен файл қабылданбайды	Шектен бір байт үлкен файлмен тест
Үзілген файл соңғы ресурсқа айналмайды	Клиентті орта жолда тоқтату
Қате сертификат атауы қабылданбайды	Қате server_hostname арқылы тест
Қатар клиенттер бірін-бірі шатастырмайды	Бірегей id бар кемінде 5 клиент
Ресурс шектері орындалады	Кезек, квота және мерзім тесттері
Қателерді талдауға дерек жеткілікті	Құпиясыз id және қате коды бар журнал

Жоба есебі және бағалау

Есепке мәселе мен пайдаланушы әрекеті, компоненттер құрылымы, хабар келісімі, іске қосу нұсқаулығы, сынақ кестесі, өлшеу нәтижесі және қалған шектеулер кіреді. «Барлық тест өтті» тұжырымының жанында қандай сценарий тексерілгені көрсетіледі. Нақты өлшенбеген жүктеме шегі немесе тұрақтылық кепілдігі жазылмайды.

Бағалау өлшемі	Ұпай	Күтілетін дәлел
Хаттама және функциялар	25	Дұрыс файл, id және қате жауаптар
Қате өңдеу және ресурстар	20	Үзілу, мерзім, кезек, квота тесттері
Қауіпсіздік	20	TLS тексеруі және қауіпсіз сақтау жолы
Тест және өлшеу	20	Автоматты сынақтар мен өлшеу кестесі
Код құрылымы және түсіндіру	15	Модульдер, нұсқаулық, қорғау жауабы
Барлығы	100	Жоба нәтижелерінің жиынтығы

Бұл кесте қорытынды жобаны бағалауға арналған ұсыныс. Пәннің ресми бағалау жүйесі оқу орнының силлабусымен анықталады. Қорғауда студент архитектурасын көрсетіп, бір дұрыс және екі ақаулы сценарийді орындайды. Одан кейін «жауап жоғалғанда не істейсіз», «қосылым саны өссе не шектейді» және «хэш неге аутентификация емес» сұрақтарына жобаға сүйеніп жауап береді.

Бақылау сұрақтары

1. Функциялық талапты тексерілетін сапа талабына қалай толықтырасыз?
2. Readiness пен liveness айырмасы қандай?
3. Реттелген тоқтатудың мерзімі неге шектеледі?
4. Жадтағы күй бірнеше серверге көшуге қалай әсер етеді?

Өздік жұмыс: қорытынды жобаны іске асырып, 5–8 бет есеп пен іске қосу нұсқаулығын дайындаңыз. Алдымен хаттама мен бір клиенттің дұрыс сценарийін бекітіңіз, кейін TLS, қатар клиенттер, квота және теріс сынақтарды қосыңыз. Соңғы көрсетілімге дейін жаңа компьютердегі таза каталогтан іске қосылуын тексеріңіз.

Дәрісті өткізу реті: талап пен архитектураға 20 минут, конфигурацияға 15 минут, өмірлік циклге 20 минут, жоба келісіміне 25 минут, бекітуге 10 минут.

Қосымша Негізгі терминдер

Термин	Қазақша түсіндірме
Socket	Процестің желілік енгізу мен шығару нүктесі
Client	Байланысты немесе сұрауды бастайтын тарап
Server	Белгілі нүктеде сұрауларды қабылдайтын тарап
Protocol	Хабар пішімі, мағынасы және алмасу ретінің келісімі
IP address	Интернет деңгейіндегі желілік мекенжай
Port	Тасымалдау деңгейіндегі қызмет нөмірі
DNS	Домендік атауларды ресурс жазбаларымен байланыстыратын жүйе
CIDR prefix	Мекенжайдың желіге тиесілі биттер санын көрсететін префикс
Loopback	Осы компьютердің өзіне желілік қатынау жолы
TCP	Реттелген байт ағынын беретін тасымалдау хаттамасы
UDP	Жеке датаграммалар тасымалдайтын хаттама
Datagram	Дербес шекарасы бар тасымалдау хабарламасы
Framing	Байт ағынын жеке қолданбалық хабарларға бөлу
Serialization	Құрылымды деректі тасымалдау не сақтау пішіміне түрлендіру
UTF 8	Unicode таңбаларын байттармен кодтау тәсілі
Byte order	Көпбайтты санның байттарын орналастыру реті
Timeout	Бір күту әрекетіне қойылған уақыт шегі
Deadline	Жалпы операция аяқталуға тиіс шекті уақыт
RTT	Хабардың барып, жауаптың қайту уақыты
Throughput	Уақыт бірлігінде сәтті өңделген дерек не сұрау көлемі
Concurrency	Бірнеше жұмыстың уақыт аралығында қабаттасып орындалуы
Thread	Процесс ішіндегі орындау ағыны
Race condition	Ортақ әрекеттер ретіне тәуелді қате нәтиже
Deadlock	Қатысушылардың бірін-бірі босатпай күтуі
Coroutine	Күту нүктесінде басқаруды бере алатын орындалу бірлігі
Event loop	Дайын оқиғалар мен міндеттерді орындайтын цикл
Backpressure	Төменгі тұтынушы үлгермегенде өндірушіні баяулату
TLS	Арнаның құпиялығы мен тұтастығын қорғайтын хаттама
Authentication	Қатысушының түпнұсқалығын тексеру
Authorization	Қатысушының әрекетке рұқсатын тексеру
Idempotency	Бірдей әрекетті қайталағанда көзделген әсердің ұлғаймауы
Backoff	Қайталау әрекеттерінің күту аралығын ұлғайту
Jitter	Қайталау уақытына басқарылатын кездейсоқтық енгізу
Hash digest	Дерек мазмұнынан есептелетін тұрақты өлшемді із
Readiness	Қызметтің жаңа сұрау қабылдауға дайындығы
Liveness	Қызмет процесінің тірі жұмысын тексеру

Қосымша Мысалдарды іске қосу анықтамалығы

Код атаулары дәріс ішіндегі мысал тақырыбында беріледі. Бірнеше мысал protocol.py модулін пайдаланады. Оқу ортасында барлық Python файлдарын бір каталогқа сақтау тәуелділікті іздеуді жеңілдетеді. Сервер командалары тоқтатылғанға дейін жұмысын жалғастырады.

Апта	Негізгі файлдар	Қажетті орта
1	week01_url.py	Python стандартты кітапханасы
2	week02_addresses.py	Жүйедегі localhost баптауы
3	week03_server.py және week03_client.py	Екі терминал, TCP 9000
4	week04_server.py және week04_client.py	Екі терминал, UDP 9001
5	protocol.py	Кейінгі мысалдарға ортақ модуль
6	week06_server.py және week06_client.py	protocol.py, TCP 9002
7	week07_server.py және week07_client.py	Python 3.12 немесе үйлесімді, TCP 9003
8	week08_client.py	Жергілікті http.server, TCP 8000
9	week09_api.py және week09_client.py	Екі терминал, TCP 8001
10	week10_sse.py	Браузер, TCP 8002
11	week11_server.py және week11_client.py	protocol.py, OpenSSL, сертификат, TCP 9443
12	week12_retry.py	Серверсіз қайталау көрсетілімі
13	week13_server.py және week13_client.py	protocol.py, файл, TCP 9004
14	test_protocol.py және week14_measure.py	protocol.py, өлшеуге 9 Дәріс. сервері
15	week15_config.py	Орта айнымалылары не әдепкі мәндер

Порт бос емес қатесі шықса, алдыңғы серверді тоқтатыңыз немесе сервер мен клиенттегі портты бірге өзгертіңіз. ModuleNotFoundError шықса, protocol.py файлының атауы мен каталогын тексеріңіз. UnicodeDecodeError шықса, кодтау келісімін және толық хабардың жиналуын тексеріңіз. Сертификат қатесін тексеруді өшірумен түзетпеңіз; оның атауын, сенімін және мерзімін салыстырыңыз.

Қосымша Бекітуге арналған қысқа жауаптар

1 апта. Клиент пен сервер — әрекеттесу рөлдері. Бір бағдарлама басқа қызметке сұрау жібергенде клиент рөлін де атқарады. Кідіріс бір әрекеттің уақытын, өткізу қабілеті уақыттағы жұмыс көлемін көрсетеді.

2 апта. /26 ішкі желіде 64 мекенжай бар. 192.168.5.77/26 мекенжайы .64–.127 блогына кіреді. Кәдімгі IPv4 ішкі желісіндегі хост аралығы .65–.126.

3 апта. accept дерек алмасатын жаңа сокет береді. Тыңдаушы сокет жаңа қосылымдарды қабылдауға қалады. Бір recv толық хабарды беруге міндетті емес.

4 апта. UDP хабар шекарасын сақтайды, жеткізуді кепілдемейді. Идентификатор кешіккен және қайталанған жауаптарды ажыратуға көмектеседі. Мекенжайды тексеру криптографиялық аутентификация емес.

5 апта. Ұзындық префиксі дене көлемін байтпен береді. Дене 300 байт болса, төрт байттық тақырыппен кадр 304 байт. JSON синтаксисі мен хабар шекарасы бөлек тексеріледі.

6 апта. Құлып ортақ күйдің құрама өзгерісін қорғайды. Желі күтуін құлып ішінде орындау басқа жұмысшыларды бөгейді. Кезек шегін өсіру сервердің өңдеу қабілетін өсірмейді.

7 апта. await күту нүктелерінде басқа міндеттерге орын береді. drain шығыс буферінің қысымын басқарады, бірақ қашықтағы бизнес әрекеттің орындалуын растамайды.

8 апта. 404 HTTP серверінің жауабы; қосылудан бас тарту кезінде HTTP жауабы жоқ. Content-Length кодталған дененің байт санына сүйенеді.

9 апта. Жадтағы күй процесс қайта іске қосылғанда жоғалады. Дерекқордағы тұрақты күй мен сервердегі жасырын клиент сеансының мағынасы әртүрлі. Әр ресурс әрекеті валидация мен рұқсатты тексереді.

10 апта. SSE бір бағыттағы оқиғаларға арналған; WebSocket екі бағыттағы хабар алмасуды береді. SSE ішінде бос жол оқиғаны аяқтайды. Қайта қосылу жоғалған тарихты жеке өзі қалпына келтірмейді.

11 апта. TLS тексеруі сенім тізбегі мен күтілетін атауды қамтиды. Сертификат клиентке беріледі, сервердің жеке кілті берілмейді. Авторизация TLS-тен бөлек орындалады.

12 апта. 3 секунд бюджетте алғашқы екі әрекет 2 секунд, аралықтар 0,6 секунд алса, соңғы әрекетке 0,4 секунд қалады. Бұл есеп басқа үстеме уақытты қоспайды; нақты код қалған уақытты сағаттан есептейді.

13 апта. SHA-256 мазмұн сәйкестігін тексереді, жіберуші тұлғасын растамайды. Уақытша файл үзілген деректің аяқталған ресурс ретінде көрінуін шектейді. Жалпы квота бөлек қажет.

14 апта. Модульдік тест парсерді бақыланатын байттармен тексереді. Интеграциялық тест нақты сокет, процесс және олардың өмірлік циклін қамтиды. Қате сұраулар өлшеу есебінен жасырылмайды.

15 апта. Readiness жаңа жұмысқа дайындығын, liveness процестің тірілігін көрсетеді. Реттелген тоқтату жаңа қабылдауды тоқтатып, ағымдағы жұмысқа шектелген уақыт береді.

Пайдаланылған дереккөздер

Дәріс мәтіндері мен оқу мысалдары осы курс үшін құрастырылған. Нөмірлі сілтемелер хаттама ережелері мен кітапхана интерфейстеріне қатысты ресми құжаттарға бағыттайды. Құжаттаманы қарау күні 2026 жылғы 14 қыркүйек. Python құжаттамасының /3/ адресі ағымдағы тұрақты тармаққа бағытталады; курс мысалдары Python 3.12 және кейінгі үйлесімді интерфейстерді қолданады.

1. Python Software Foundation. socket — Low level networking interface. https://docs.python.org/3/library/socket.html

2. Python Software Foundation. Socket Programming HOWTO. https://docs.python.org/3/howto/sockets.html

3. IETF. RFC 9293. Transmission Control Protocol TCP. 2022. https://www.rfc-editor.org/rfc/rfc9293.html

4. IETF. RFC 768. User Datagram Protocol. 1980. https://www.rfc-editor.org/rfc/rfc768.html

5. IETF. RFC 9110. HTTP Semantics. 2022. https://www.rfc-editor.org/rfc/rfc9110.html

6. IETF. RFC 1034. Domain Names Concepts and Facilities. 1987. https://www.rfc-editor.org/rfc/rfc1034.html

7. IETF. RFC 8200. Internet Protocol Version 6 Specification. 2017. https://www.rfc-editor.org/rfc/rfc8200.html

8. Python Software Foundation. asyncio Streams. https://docs.python.org/3/library/asyncio-stream.html

9. Python Software Foundation. http.server — HTTP servers. https://docs.python.org/3/library/http.server.html

10. Python Software Foundation. urllib.request — Extensible library for opening URLs. https://docs.python.org/3/library/urllib.request.html

11. Python Software Foundation. threading — Thread based parallelism. https://docs.python.org/3/library/threading.html

12. Python Software Foundation. queue — A synchronized queue class. https://docs.python.org/3/library/queue.html

13. Python Software Foundation. unittest — Unit testing framework. https://docs.python.org/3/library/unittest.html

14. IETF. RFC 6455. The WebSocket Protocol. 2011. https://www.rfc-editor.org/rfc/rfc6455.html

15. IETF. RFC 8446. The Transport Layer Security Protocol Version 1.3. 2018. https://www.rfc-editor.org/rfc/rfc8446.html

16. IETF. RFC 9000. QUIC A UDP Based Multiplexed and Secure Transport. 2021. https://www.rfc-editor.org/rfc/rfc9000.html

17. Python Software Foundation. ssl — TLS SSL wrapper for socket objects. https://docs.python.org/3/library/ssl.html

18. IETF. RFC 8259. The JavaScript Object Notation Data Interchange Format. 2017. https://www.rfc-editor.org/rfc/rfc8259.html

19. IETF. RFC 8085. UDP Usage Guidelines. 2017. https://www.rfc-editor.org/rfc/rfc8085.html

20. WHATWG. HTML Living Standard. Server sent events. https://html.spec.whatwg.org/multipage/server-sent-events.html

21. Python Software Foundation. logging — Logging facility for Python. https://docs.python.org/3/library/logging.html

22. Roy Thomas Fielding. Architectural Styles and the Design of Network based Software Architectures. Chapter 5. 2000. https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

23. Python Software Foundation. hashlib — Secure hashes and message digests. https://docs.python.org/3/library/hashlib.html

24. Python Software Foundation. struct — Interpret bytes as packed binary data. https://docs.python.org/3/library/struct.html

25. Python Software Foundation. Coroutines and Tasks. https://docs.python.org/3/library/asyncio-task.html

26. Python Software Foundation. ipaddress — IPv4 IPv6 manipulation library. https://docs.python.org/3/library/ipaddress.html

27. Python Software Foundation. urllib.parse — Parse URLs into components. https://docs.python.org/3/library/urllib.parse.html